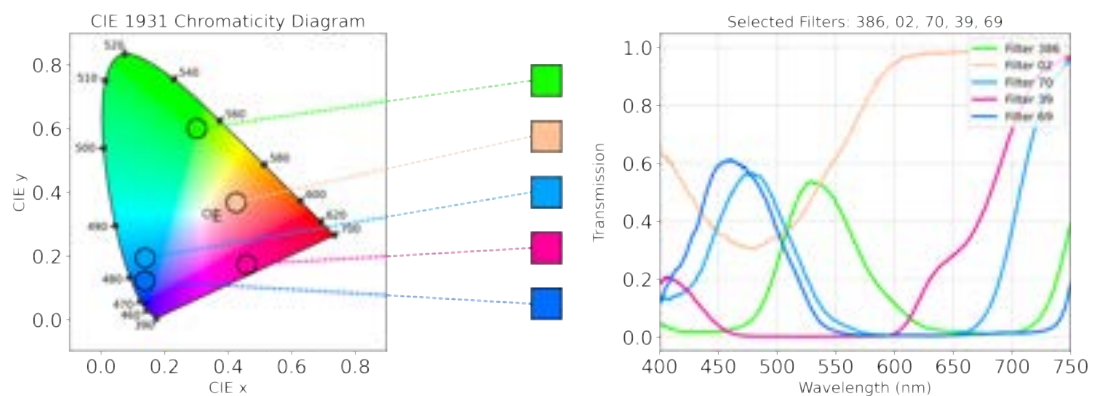


Cambridge University MPhil Advanced Computer Science

Filter Wheel Design for Optimal Color Reconstruction



Author:

Kyle Fram

Lucy Cavendish College

Date: June 22, 2026

Supervisors:

Rafal Mantiuk

Graham Finlayson

Declaration

I, Kyle Fram of Lucy Cavendish College, being a candidate for the Master of Philosophy in Advanced Computer Science, hereby declare that this report and the work described in it are my own work, unaided except as may be specified below, and that the report does not contain material that has already been used to any substantial extent for a comparable purpose. In preparation of this report, I adhered to the Department of Computer Science and Technology AI Policy. I am content for my report to be made available to the students and staff of the University. I have used AI tools in the preparation of charting utilities, as well as some organizational utilities in my repository. Specifically, modules in the `/vis/` subfolder of the repository, as well as `schedule.py`, the scheduling utility I used to run experiments automatically, and further `charting.py`, the library used for creation of figures and graphics, were written with the help of Claude Sonnet 4.6. Additionally, the same model was used for some readability edits of the codebase, such as variable renaming.

Signed: Kyle Fram

Date: June 22, 2026

Word Count

Using `texcount -inc -sum main.tex`, the total word count of this report is 11763. The page count excluding frontwork, includes page 2-46, for a total of 45 pages.

Abstract

A camera’s color accuracy is limited by the selection of color filters and on the combination weights for reconstruction of each color channel, which is ill-conditioned in many digital cameras. We cannot ordinarily change filters in a standard digital camera, but the filter wheel camera construct allows for easy replacement of filters and adjustment of combination weights. We propose a filter selection and weight calculation methodology that provides high quality color reconstruction using as few as 5 color filters, at a low cost. First, we simulate the selection and reconstruction process using a simulated dataset of filters, before applying the same methodology to our set of gel filters, for which we measure each spectral transmission function. Then, we refine the filter selection process using an improved algorithm, wherein we combine Matching Pursuit methods with an exhaustive search over all filter combinations. We find that the Matching Pursuit combined with a few rounds of exhaustive search can find near-optimal solutions in all conditions. Once the filters are selected, we benchmark our simulated capture and reconstruction pipeline with CIELAB ΔE_{00} , a human-perception based color similarity metric. We consistently achieve simulated ΔE_{00} below 1 for various human eye color matching functions, which indicates color differences generally below the threshold of human detection. Then we implement the camera using a microcontroller, monochromatic camera, 3D printed components, and a set of gel filters. This camera is deployed in a lab environment, and we attempt to reconstruct color informed by our simulated filter selection methodology. Physical reconstruction demonstrates the viability of the approach, though lab results reveal practical challenges, including sensor noise and illumination instability, that degrade performance relative to simulation. Code related to filter selection, ΔE_{00} computation, data gathering, data visualization, and other utilities is available at <https://github.com/kylefram/MultispectralCamera/tree/main>.

Chapter 1

Introduction

Multispectral image acquisition systems have uses ranging from agricultural monitoring to fine art analysis to medical scanning [1, 2, 3, 4, 5, 6, 7, 8]. State of the art systems are prohibitively costly [9], and are generally only suitable for their specialized purpose. Color photography first proliferated in the twentieth century, with filters employed to discriminate spectral bands, then using a layering process to recombine images into color [10]. A modern, flexible solution lies in the filter wheel, notably employed in terrestrial and geological analysis as early as 1972 [11], and now provides a useful framework for multispectral imaging. In this paper, we construct an effective system using simple 3D-printed components, microcontrollers such as the Raspberry Pi, and a set of inexpensive gel filters such as those used for stage lights.

However, our aim is to improve upon the classic a-priori fixed filter wheel used with a monochromatic camera. Here, we simulate image capture through a set of colored filters and then combine these to best approximate the desired XYZ colorimetric values, effectively solving for a new filter wheel.

Prior work has been limited by computational resources, and generally relies on heuristic measures to select filters [12, 13]. We provide a minimal error reconstruction for the set of Rosco Supergel [14] filters.

We construct a simulated camera using a Python-based infrastructure. The model aims to replicate the individual parts of the physical camera, namely the monochrome image input, the filter overlays, the image combination logic, and the processed output. The simulation also includes modules to perform the filter selection task under different conditions and compare results of filter selection methodologies. This comparison relies upon a loss metric defined to guide the filter selection, and a ΔE_{00} (CIEDE2000) measure to compare reconstruction results of filter selection. We also output spectral image simulations for a qualitative comparison of our images.

In section 3.5, we show that we are able to consider a larger set of filters by deploying our filter selection method and simulated camera on multiple GPUs and parallelizing our search. Further, we examine the reconstruction quality as a function of k filters included in our filter wheel, as well as the tradeoff between reconstruction quality and filter selection runtime of various heuristic filter selection algorithms.

We provide a framework for filter selection for any target spectral response function. By generalizing our target function, we allow for applications beyond RGB reconstruction, such as hyperspectral and multispectral imaging, or even non-imaging applications such as spectral sensing. We also explore the performance of our system under different human eye color matching functions, such as CIE 1931 [15] and CIE 2006 LMS functions [16]. In the analysis of human eye color matching functions, we approach Luther Condition Compliance, which is the condition that a camera’s spectral sensitivity functions are a linear transformation of the human eye’s color matching functions [17].

In section 4.2 we show that our methodology provides accurate reconstruction by evaluating the 24 Macbeth Color Checker patches and the Preferred Memory Color Chart (PMCC), computing ΔE_{00} under standard illuminant for the CIE 2006 XYZ target functions. We compute the relationship between number of filters k and minimum ΔE_{00} .

We capture images in a controlled luminance setting using the Arducam 1MP OV9281/OV9282 Global Shutter Camera [18] and report on the results of different k number of filters. We are able to evaluate the simulated selections in a real world environment, albeit a controlled lighting environment, again providing a comparable evaluation compared with our simulated spectral dataset provided by the CAVE dataset. The camera captures a scene including a Preferred Memory Color Chart Reader, an improvement on the Macbeth color checker patches, and we use this as a baseline to evaluate the quality of our deployed system.

Understanding that camera capture varies significantly in differing lighting conditions, we incorporate a noise model in our framework. We develop an addendum to our loss function framework which incorporates camera noise in the reconstruction quality and allows us to vary the filter set based on expected capture noisiness. Conceptually, this noise is a function of photonic arrival into our camera sensor, motivating our analysis over varying brightness setups, or equivalently, exposure times.

In the remaining sections of this paper, we describe our methodology, from data collection to mathematical formalization to designing an optimized filter selection procedure. Next, we evaluate the performance of our procedure in context of our loss function as well as ΔE_{00} , for the CIE 1931 and CIE 2006 XYZ standard observers. We examine our fully simulated framework, as well as piecewise introducing real-world components, starting with the Rosco Supergel filter set and then moving to cameras and then evaluating the entire physical camera stack. We introduce a noise term to our loss function in order to model the tradeoff between exposure time and capture quality as well as to conceptualize the effect of scene luminance on capture quality. We conclude our filter selection section with a discussion on the tradeoff between timing and performance for various use cases.

Chapter 2

Related Work

Our work combines concepts from color science, optics, light science, and camera hardware to construct a full camera capture hardware and software stack. We build upon a vast preexisting literature on the multispectral filter wheel schematic, with notable implementations dating back to 1972 and prior with NASA’s Landsat 1 [11, 19] multispectral geological imaging system. Advances in camera, microcontroller, and 3D printing technologies enable us to deploy a relatively cheap multispectral filter wheel camera, which we claim can replicate results of state of the art cameras, albeit hindered by slow image capture speeds. Our platform can also be deployed across a range of functional use cases, and while we focus on the replication of the human eye response function, we develop our methodology with the understanding that our platform may be used for other multispectral use cases with a quick replacement of some filters. Therefore, we include a section of broad multispectral imaging applications for which we may deploy our camera.

2.1 Filter Wheel Design

The spinning filter wheel construct provides an inexpensive solution when unconstrained by capture duration, such as in the imaging of fine art and agriculture. Initial formulations used bandpass filters, an oscillating scan mirror, and a spinning filter wheel for satellite-mounted geological surveys [20]. Koenig and Praefcke [21] design an early prototype of a multispectral scanner based on a filter wheel, and compare the performance of varied reconstruction methodologies.

Hardeberg [12] devises and deploys a multispectral filter wheel apparatus, including a section on filter selection methodology. This work also provides a benchmark for our system’s ΔE_{00} and error results, which we will elaborate upon in section 3.

2.2 Filter Selection

While nearly all the multispectral image acquisition works referenced in this section mention their filter selection methodologies, we attempt to improve upon those by borrowing a few novel techniques adapted from other applications. As we will discuss in section 3.3, we must apply a weighting factor to each filter in the reconstruction of each target function. Under certain conditions and decisions regarding loss functions which we will elaborate in 1, the pseudoinverse [22, 23, 24] provides a stable, and in fact verifiably optimal set of weights. We use the above result as a branching start point for solutions under different conditions, specifically in our analysis of heuristic algorithms to find optimal weights such as Matching Pursuit based solutions [25] or LASSO based solutions [26, 27].

2.3 Filter Transmission Function Evaluation

High-quality filter-based multispectral imaging systems rely upon works in filter technologies and their respective quality and transmission metrics such as those derived by Vora, Trussell, and Vrhel [13, 28, 29]. Ng and Allebach [30] develop sufficient and necessary conditions for recovering accurate measurements of spectral reflectances and tristimulus values using a set of filters and the protocol for design of these filters. Further advances in filter design include those done by Finlayson et al. [31].

2.4 Image Response

Much of the existing literature on simulation of color matching functions relates to the CIE XYZ models of the human eye. We follow the work of Gao et al. [32] in matching our simulated camera’s spectral sensitivity function to the human eye by optimizing for a color correction matrix M best matching the set of filtered camera sensitivities to the XYZ color matching functions. This method achieves optimality only when we approach Luther Condition compliant capture [33], specifically that our spectral sensitivity functions be linearly related to our chosen XYZ color matching function. We follow the work of Finlayson and Zhu [31] in their analysis of Luther-condition compliant filters. However, rather than designing a specialized filter, we investigate whether we can approach Luther compliance by adding incrementally more generic filters to our filter wheel.

2.5 Noise Model

While most of our laboratory experiments are done using D65 lighting, we design the system to be robust to other lighting environments. To this end, we must define the effect of camera noise as a function of lighting and exposure time, for which we rely upon work by Hanji et al. [34] to derive a simple Poisson noise estimator. In general, high dynamic range image systems [35, 36, 37] have well defined noise estimation systems for multiple exposure image capture systems, providing a useful benchmark for our work. Sidiropoulos [37] also specifically studies the effect of filtering systems and their noisiness with regards to multiple exposures.

2.6 Multispectral Imaging Applications

Lapray et al. [38] provide an overview of multispectral imaging technologies as well as their costs, providing a cost benchmark for our work. Morales et al. [39] demonstrate a bandpass filter wheel used for precision agriculture applications, and Scutelnic et al. [9] propose a similar filter wheel construct to build a relatively low cost sensor array.

Given the static nature of the subject matter and necessity for accurate spectral readings, multi-filter image acquisition systems are very prevalent in fine art analysis. Early works before 2000 pioneered the technologies such as Martinez et al. [40], devising a scanner with integrated lighting which could be placed above a 1 meter by 1 meter painting, capturing images through a filter wheel under controlled luminance. Yokoyama et al. [41] quantify the ΔE performance of a similar system using a color wheel on a tripod instead of a scanner affixed to the painting, finding comparable visual results to the scanner.

In addition to the satellite-mounted implementations above [11, 19], multispectral imaging platforms have been deployed for geological surveys such as those enumerated in a survey of multispectral lithological geography [1]. Other aerial mounted systems and applications include unmanned aerial vehicle (UAV) mounted agricultural scanning systems which serve to monitor crop growth and pest prevalence [9, 8]. These applications provide solutions to the low shutter

speed of the filter wheel construct, requiring simultaneous capture of each filter in satellite-mounted systems [42] or post-capture image correction and synchronization in UAV mounted systems [43].

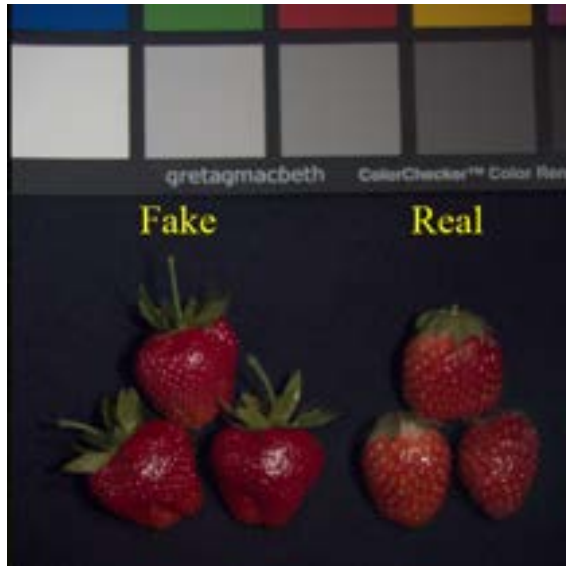
Mariotti et al. [3] summarize hyperspectral and multispectral imaging use cases in crime scene analysis from 2011-2021, another example of (hopefully) static scenes appropriate for analysis with our tool and its predecessors [44, 45, 46].

2.7 External Datasets and Tools

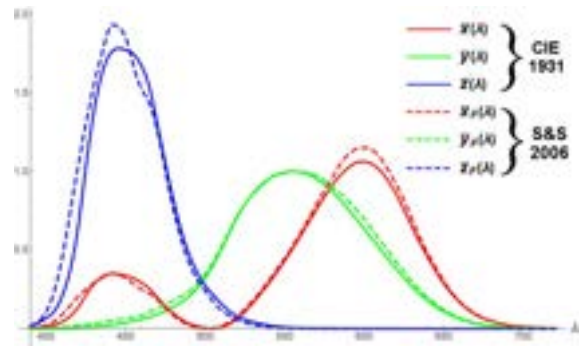
While the main body of research relies on a usable, portable camera, we use some external datasets to inform our development and analysis.

2.7.1 Spectral Reconstruction

We use the CAVE spectral dataset [47] for our simulated spectral reconstructions. This dataset consists of multispectral images including one ground truth capture per scene and 31 captures of reflectance data over 10 nanometer (nm) steps between 400 and 700nm. Subjects include food, human skin and hair, assorted items, and a few 'real and fake' side by side captures of items such as food and models with different reflectance characteristics.



(a) 'Fake & Real' Image from CAVE Dataset



(b) CIE 1931 and S&S 2006 Spectral Sensitivity Functions

Figure 2.1: External datasets used for evaluation and simulation.

2.7.2 XYZ Functions

The UCL Computer Vision and Research Laboratory (CVRL) publishes various data sets relevant to color research [48, 16]. This site serves as a repository for various spectral sensitivity functions of the human eye, from the standard CIE 1931 trichromatic functions, to Stockman & Sharpe's revised 2000 and 2006 versions, to other useful datasets such as Stiles & Burch 1959 2° and 10° trichromator measurements.

2.7.3 X-Rite/Macbeth Color Checker

While not specifically a dataset per se, the X-Rite color checker provides 24 patches for which to create standard color profiles for cameras and displays [49, 50, 51]. This set was developed

in 1976 and has been the industry and research standard of color calibration for much of its existence. The evaluation of ΔE_{00} and the approximate scale we use to evaluate color quality differences has been developed by Lindstrom [52].



(a) X-Rite Color Checker



(b) PMCC Grid

Figure 2.2: Color chart references used for calibration and evaluation.

2.7.4 Preferred Memory Color Checker

We also use a Preferred Memory Color Checker [53, 54] for calibration as well as analysis of our color capture. This improves upon the 24 patches of the Macbeth Color Checker often used by prior works in the space, allowing us to record more accurate ΔE_{00} values. This color checker includes 30 patches, optimized for human-centered perception, rather than the 24 balanced patches of the Macbeth color checker.

Chapter 3

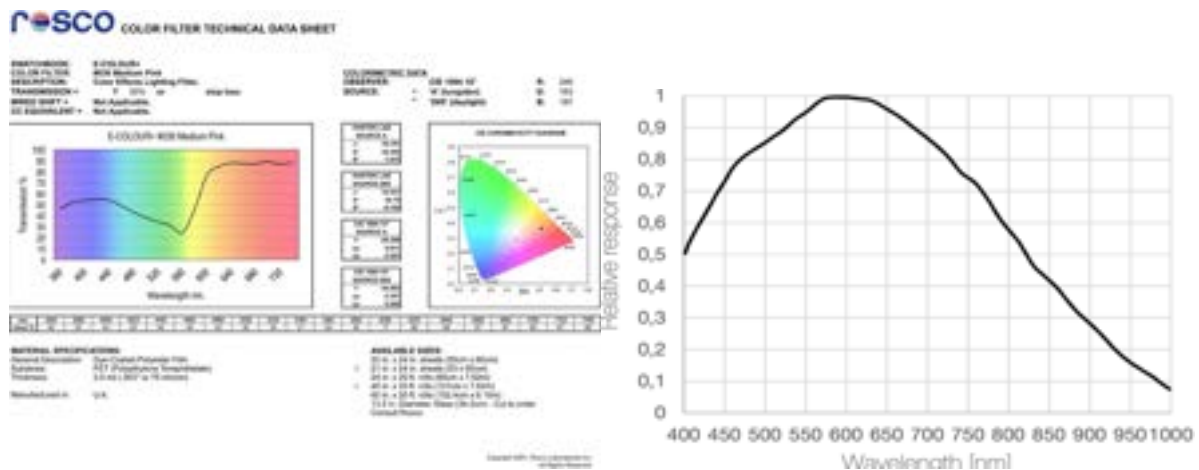
Method

The following section outlines the structure of the project, from theoretical formulation, to data acquisition, to lab testing, to deployment. The camera pipeline is first developed in a simulated environment, with spectral transmission functions acquired from producer specifications and camera sensitivity functions acquired in the same way. Thus, a full simulation of our camera can be deployed on multispectral image sets and tested in our filter selection harness. Later, measurements of a subset of filters are taken in the lab.

3.1 Data Collection

The aim of our method is to capture multiple images through different color filters and then combine them to match the colors of an image of a scene captured with respect to the CIE XYZ color matching functions (CMFs). We will choose filters drawn from a set of Rosco Supergel filters that we measure in our lab. We model our camera's and optional IR-cutoff filter's spectral response function as per the manufacturer's specifications [18, 55]. All filters are represented in the domain $\omega = [400, 750]$ nm, with $\lambda \in \omega$.

First, we gather a set of 494 filters from the Rosco Supergel MyColor tool [14]. Rosco provides a specification sheet and spectral response function for each filter in the MyColor catalog.



(a) Example filter specification sheet from MyColor (b) Camera sensitivity function for $\lambda \in [400, 1000]$

Figure 3.1: Scraped and measured spectral transmission functions

We use a similar approach for our simulated camera, with specifications acquired from Arducam [18]. The listed specification sheets provide charts defining camera response over ω_d .

The simulated set of 494 filters from the MyColor tool cover a large space, useful for finding a best-case-scenario when nearly unconstrained by transmission function. In reality, we have a set of 119 filters to choose from, which will be sufficient for practical results. We use a dark room, Lilliput lamp [56], and a spectrophotometer to measure the transmission functions of our set of filters.



(a) Set of 119 Supergel filters

(b) Spectrophotometer and Spectralon

Figure 3.2: Filter set, Spectrophotometer setup and measurement workflow

We use a stage consisting of an overhead illuminant, a spectralon placed at a 45° angle, and a spectrophotometer pointed at the stage, pointing 45° below the horizontal plane for alignment with the spectralon. The spectrophotometer takes a radiance reading at each wavelength from 350-1000. For this case, we are interested in measuring the transmission function of each filter. We can compute the x and y coordinates from the spectral radiance of each measurement.

Brightness in the room was controlled by covering the walls with black felt and ensuring that the computer monitor was pointed away from the stage. The spectrophotometer is equipped with a laser to indicate measured surface, and each filter was placed flush against the spectralon for measurement.



Figure 3.3: Computer readout of spectrophotometer measurement session

The stage and illuminant can be seen on the left above, with the control panel and transmission function readout on the right. The results of each scan are saved into a *.mat* file at 1 nanometer increments, along with a few calibration runs with no filter present.

For our target functions, we use the UCL Color & Vision Research Laboratory repository of cone and rod sensitivity functions [48], including CIE XYZ 1931 2^o and CIE 2006 XYZ 2^o functions.

We also benchmark our performance by computing ΔE_{00} using the RGB coordinates of the Macbeth ColorChecker following Pascale [49]. For a demonstration of color reconstruction across scenes, we use the CAVE dataset from Yasuma et al. [47]



(a) Items staged for color capture, including a PMCC (b) Lab setup for multispectral imaging

Figure 3.4: Spectrophotometer and laboratory scene setup.

Next, we replicate the above simulated spectral reconstruction in the lab, capturing a scene including one Preferred Memory Color Chart [54] under the same Lilliput illuminant [56].

To this end, each filter was placed in front of the camera, mounted onto a tripod and aimed at a scene including objects of different reflectances and colors. The PMCC was also placed at the front of the scene for easy quantitative color analysis. The camera was kept in place such that we could pick one pixel per capture for which to measure each color patch under each respective filter.



(a) Capture of filter #398, including PMCC (b) Subset of items staged for color capture

Figure 3.5: Example captures used for qualitative evaluation.

Ground truth measurements were taken with the same spectrophotometer as above, under consistent angular setup and the same lighting. Each panel of the SPD was captured in order to compute absolute colorimetric difference between spectral reconstruction and ground truth.

A render of the readings taken with the spectrophotometer is shown, providing a human-eye check that our readings were taken correctly. The white patch provided a white point and other patches were rendered after normalizing to this white, taking the illuminant as D65.

We plot the color patches on the standard 1931 XYZ color gamut for illustration.

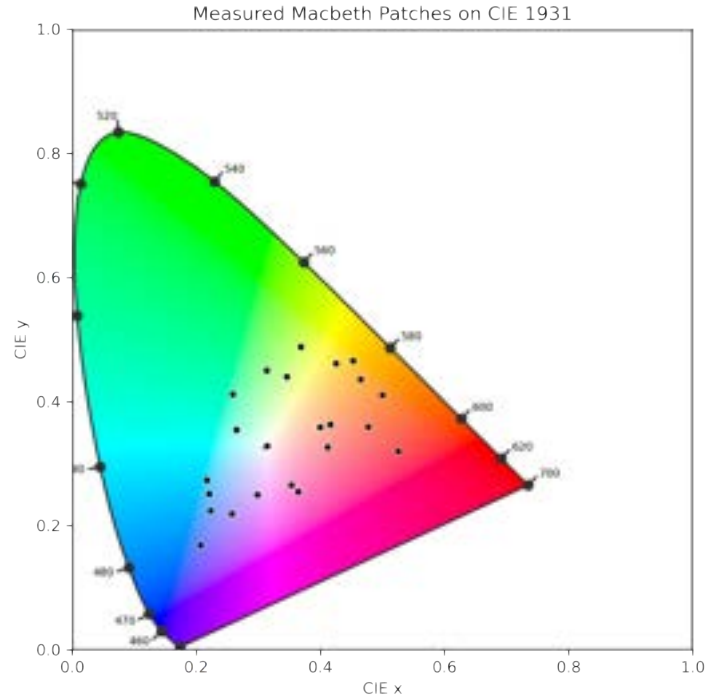


Figure 3.6: Distribution of patch locations on the CIE 1931 color gamut

3.1.1 Captures and Measurements

We present the lab captures and corresponding spectrophotometer transmission measurements for each filter used in the final simulation set. Each pair shows the monochrome RaspberryPi capture (left) alongside the measured transmission spectrum (right).

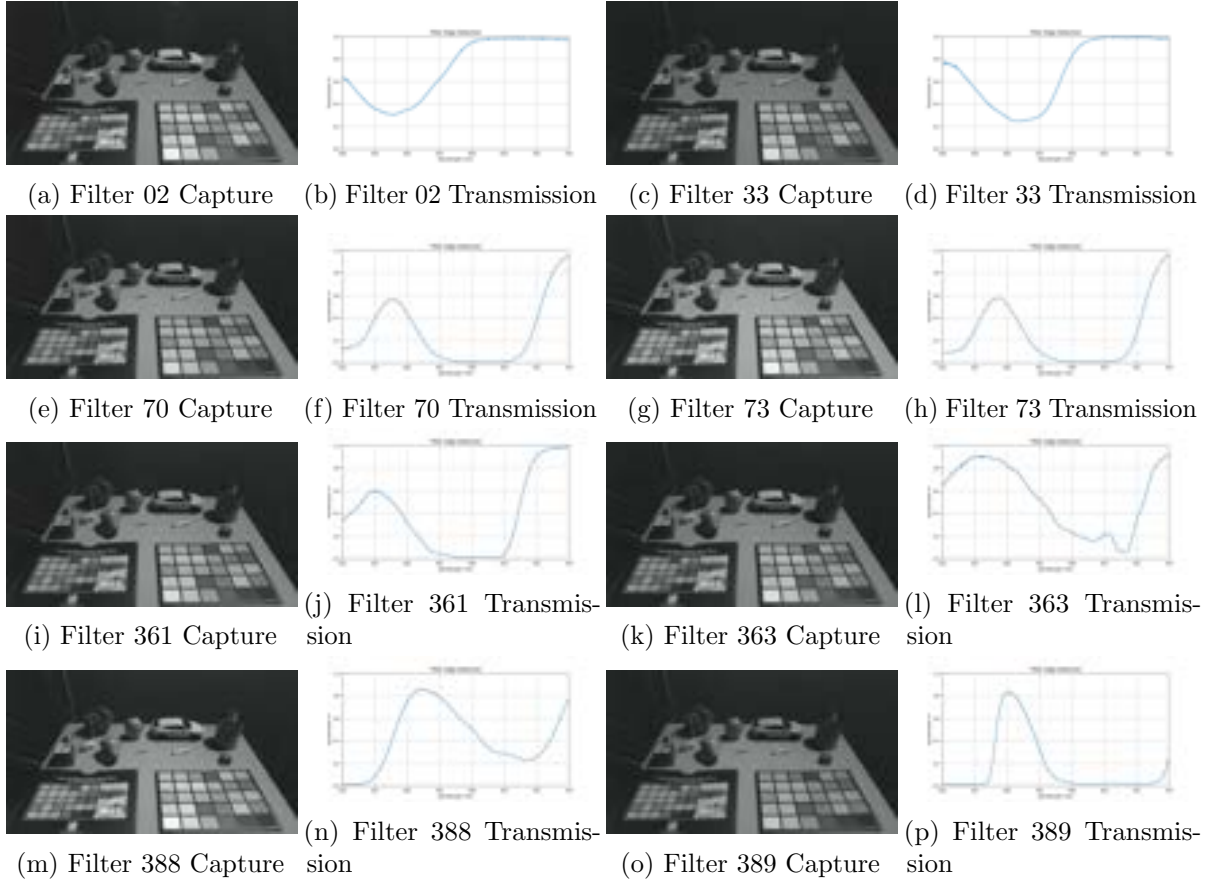


Figure 3.7: Lab Captures and Transmission Measurements for Each Filter

3.2 Color Formation Model

In our model, we have a single grey-scale camera (only one channel). Thus, our estimated new camera sensitivities are a function of the intrinsic sensitivities of this single-channel camera multiplied by the filter transmittance. Let $f_i(\lambda)$ respectively denote the i 'th filter's spectral transmission function, and let $m_{c,i}$ denote the transformation weights for combining the i 'th filter into color channel c (e.g. to best approximate the XYZ color matching functions). Then the effective spectral camera sensitivity function θ_c can be written as:

$$\theta_c(\lambda) = \sum_{i \in I} m_{c,i} f_i(\lambda) \sigma(\lambda) \quad (3.1)$$

where I denotes the set of k indices from our set of N filters and $\sigma(\lambda)$ denotes the sensor's spectral sensitivity function.

Table 3.1 summarizes the key mathematical symbols, functions, and expressions used in the problem formulation above.

In the optimizations that follow, we will solve for the R, G and B sensors that best approximate the actual X, Y and Z color matching function sensitivities, respectively referred to as $x_c \in \{X, Y, Z\}$.

3.3 Problem Formulation

For approximations of XYZ, we follow the work of Hardeberg [12] in adopting MSE loss between target and simulation over the visible spectrum as the loss function and minimizing loss via

search for an optimal $\mathbf{M}$, the $3 \times k$ matrix of $m_{c,i}$ transformation weights. For each color channel c we have MSE loss L_c as:

$$L_c = \int_{\omega} \|x_c(\lambda) - \theta_c(\lambda)\|_2 d\lambda \quad (3.2)$$

Given that we are limited by 1 nanometer steps in measurement, we discretize over ω , taking $\omega_d = \{400, 401 \dots 750\}$, functions evaluated over the discrete domain are denoted with bracket notation, e.g. $\theta_c[\lambda]$, evaluated over discrete domain. We take *MSE* loss over ω_d to compute our total loss per color channel:

$$L_c = \frac{1}{|\omega_d|} \sum_{\lambda \in \omega_d} \|x_c[\lambda] - \theta_c[\lambda]\|_2 \quad (3.3)$$

We consider loss from all color channels equally, and minimize $\sum_c L_c$ by choosing $\mathbf{M}$ for a set of I ‘active’ filters, each having transmission functions $f_i[\lambda]$ over ω_d , thus comprising a k length vector of functions. Omitting the constant $\frac{1}{|\omega_d|}$, we have for each set of filters I :

$$\arg \min_{\mathbf{M}} \sum_c \sum_{\lambda \in \omega_d} \|x_c[\lambda] - \sum_{i \in I} m_{c,i} f_i[\lambda] \sigma[\lambda]\|_2 \quad (3.4)$$

We define $\mathbf{F}$ as the stacked and discretized $k \times 351$ matrix of $f_i[\lambda] \sigma[\lambda]$ measured at each $\lambda \in \omega_d$. And the discretized set of 3×351 XYZ functions as $\mathbf{X}$.

When using a least-squares loss L_d , we can solve for $\mathbf{M}$, in closed form, using the Moore-Penrose pseudo-inverse [22, 23], denoted A^+ for a matrix A . The unique $351 \times k$ Moore-Penrose pseudo-inverse solution $\mathbf{F}^+$ has the desirable property that while we may not exactly satisfy the Luther condition, it provides the closest possible approximation for a given set of filter indices I [24]:

$$\mathbf{M} \approx \mathbf{X} \mathbf{F}^+ \quad (3.5)$$

Finally, as a point of detail we must alter this optimization if any of our filters have 0 transmission in a part of the assumed wavelength interval. The IR-cutoff filter only has nonzero transmittance in $[400, 650]$ nanometers. So, when this filter is used we can only optimize within this wavelength interval, and thus we redefine $\omega_d = \{400, 401, \dots 650\}$.

Further, we consider applications using loss functions L and their respective L_d which may not be the MSE loss. We consider SMAPE and log-loss functions amongst others, for which analytical solutions do not necessarily exist. For these, we use a gradient descent algorithm with initial guess informed by the Moore-Penrose pseudoinverse.

As an extension of the above, we model the noise of our camera using a modified MSE loss plus an error term. Following Hanji [34], we model camera photon registration as a Poisson process. Borrowing the terminology, we take the radiance at each pixel p as $\phi(p)$, having pixelwise radiance defined as $R(p) \sim \text{Pois}(\phi(p)t)$. Then, with gain g and exposure time t , the expected scene radiance and variance, or noise, are determined by

$$E(R(p)) = \phi(p)tg \quad (3.6)$$

$$V(R(p)) = E(R(p))g \quad (3.7)$$

Thus, our photonic camera noise, $V(R(p))$, is a linear function of exposure time. We provide a study in reconstruction quality as a function of ‘shutter speed,’ which refers to the duration of capture per filter. This is again implemented by defining $L = \text{MSE} + V(R(p))$, which can be achieved with a few steps of gradient descent from the MSE pseudoinverse solution.

We enumerate the loss functions supported below. Each of these loss functions is applied along the discretized wavelength domain $\lambda \in \omega_d$. The mathematical formulas are specified in 1.

1. MSE

This loss function calculates the mean squared error at each 1nm increment. We find that not only does the MSE loss function result in the best estimates for ΔE_{00} , but it has the best performance with no gradient descent steps required due to the pseudoinverse optimality condition from 3.4.

2. sMAPE

The symmetric Mean Absolute Percentage error instead compares the percentage difference at each 1nm point interval. Given some values at or near 0, this loss function often provides very good reconstruction near the 'tails' of the sensitivity distribution, but may significantly deviate from the target function near the modes of each curve. This loss function has some physiological basis following Yang et al. [57], but does not produce good results in our procedure.

3. log

This loss function has slightly different behavior vs. sMAPE given its behavior around 0, resulting in extreme deviations in the most consequential areas.

4. MSE-noisy

This adjustment to the MSE loss function takes an additional parameter t , in order to model sensor exposure time. In adding a term to the loss function, we break the pseudoinverse optimality condition from 3.4, but we approach the solution from a few rounds of gradient descent, especially with larger t , and therefore lower noise term.

Our implementation uses `pyTorch` operations in order to effectively step our gradient descent algorithm for non-MSE loss functions, as well as seamless integration into our parallelization and gradient descent pipeline for non-MSE loss (where no descent steps are needed from pseudoinverse solution).

Algorithm 1 Loss Computation with Noise-Weighting Options

Require: Reconstructed sensitivities $\theta_c[\lambda]$, Target CMFs $x_c[\lambda]$ (for $c \in \{R, G, B\}$ and $\lambda \in \omega_d$); Loss function metric L ; Transformation weights $m_{c,i}$ ($\mathbf{M}$); Filter-sensor matrix rows $f_i[\lambda]\sigma[\lambda]$ ($\mathbf{F}$); Photon budget limit P ; Noise penalty weight w_{noise} .

Ensure: Computed scalar loss value L .

```

1: Set  $\epsilon \leftarrow 10^{-6}$ .
2: if  $loss = \text{'smape'}$  then
3:    $L \leftarrow \sum_c \sum_{\lambda \in \omega_d} \frac{|\theta_c[\lambda] - x_c[\lambda]|}{|\theta_c[\lambda]| + |x_c[\lambda]| + \epsilon}$ 
4: else if  $loss = \text{'log'}$  then
5:    $L \leftarrow \sum_c \sum_{\lambda \in \omega_d} \left( \frac{\log(\theta_c[\lambda] + \epsilon)}{\log(x_c[\lambda] + \epsilon)} - 1 \right)^2$ 
6: else if  $loss = \text{'mse'}$  then
7:    $L \leftarrow \sum_c \sum_{\lambda \in \omega_d} (\theta_c[\lambda] - x_c[\lambda])^2$ 
8: else if  $loss = \text{'mse_noisy'}$  then
9:    $L_{\text{base}} \leftarrow \sum_c \sum_{\lambda \in \omega_d} (\theta_c[\lambda] - x_c[\lambda])^2$ 
10:  if  $\mathbf{F}$  is None or  $\mathbf{M}$  is None then
11:    return  $L_{\text{base}}$ 
12:  end if
13:   $L_{\text{penalty}} \leftarrow \sum_c \sum_{\lambda \in \omega_d} \sum_{i \in I} \left( m_{c,i}^2 \cdot \frac{\max(f_i[\lambda]\sigma[\lambda], 0)}{P} \right)$ 
14:   $L \leftarrow L_{\text{base}} + w_{\text{noise}} \cdot L_{\text{penalty}}$ 
15: end if
16: return  $L$ 

```

3.4 Mathematical Terms

Expression	Definition
λ	Wavelength (in nanometers).
ω	Continuous wavelength interval of interest ($[400, 750]$ nm).
ω_d	Discrete wavelength set, ($\{400, 401, \dots, 750\}$).
c	Color channel for the reconstructed sensors; $c \in \{R, G, B\}$
i	Filter index (one filter from the candidate set).
I	Set of selected (active) filter indices, with $ I = k$.
k	Number of selected filters.
$x_c(\lambda)$	Target CMF for channel c , continuous form.
$x_c[\lambda]$	Target CMF for channel c , discrete form.
$f_i(\lambda)$	Spectral transmission of filter i , continuous.
$f_i[\lambda]$	Spectral transmission of filter i , discrete.
$\sigma(\lambda)$	Camera sensor spectral sensitivity; continuous.
$\sigma[\lambda]$	Camera sensor spectral sensitivity; discrete.
$m_{c,i}$	Transformation weight for filter i in reconstruction of channel c .
$\mathbf{M}$	$3 \times k$ matrix of weights $m_{c,i}$ (color-correction matrix).
$\theta_c(\lambda)$	Effective reconstructed spectral sensitivity of channel c , continuous.
$\theta_c[\lambda]$	Effective reconstructed spectral sensitivity of channel c , discrete.
L_c	Loss for channel c .
$\mathbf{A}^+$	Moore-Penrose pseudo-inverse of matrix $\mathbf{A}$.
$\mathbf{F}$	Stacked, discretized filter-sensor matrix ($k \times 351$) with rows $f_i[\lambda]\sigma[\lambda]$ over ω_d .
$\mathbf{F}^+$	Pseudo-inverse of $\mathbf{F}$ used for least-squares fitting.
$\mathbf{X}$	Discretized target CMFs as a 3×351 matrix over ω_d .
g	Sensor gain.
t	Sensor exposure time.
$Pois(x)$	Poisson model with parameter x
$R(p)$	Poisson process representing photon arrival frequency
$E(R(p))$	Mean of the registered photon count at pixel p .
$V(R(p))$	Variance (noise) of the registered photon count at pixel p .

Table 3.1: Summary of mathematical terms.

3.5 Filter Selection

While the number of filters N may be large, we ideally would posit a filter wheel with fewer filters. Some works, such as Finlayson and Zhu [31] aim to construct optimized filters for camera and illumination specifics. However, off-the-shelf gel filters provide a cost effective tool, if a good selection can be made. We take k , the number of filters chosen from the N sized universe, as well as L_d , the discretized loss function defined above.

The problem of filter selection and $\mathbf{M}$ computation are closely related. For non-brute force methods of filter selection, we effectively 'guess' the combination that might produce the best MSE after choosing k . This lends itself to either exclusion or inclusion algorithms, the former of which we refer to as LASSO, the latter as PCA.

3.5.1 Brute Force Search

In the naive solution, we find the optimal filter combination for every subset of k filters (where k is small) drawn from the N filter set by selecting the combination with the lowest loss, considering

all combinations of filters (there are nearly 10,000,000 of these for $k = 5$ in the full Rosco Supergel set). In Figure 2 we plot the errors across all sets of 5 filters (multiplied by the camera sensitivity) from the worst to best performing quintuple. We compute the Moore-Penrose pseudo-inverse for each set of filters. Our implementation is parallelized over 8 NVIDIA L4 GPUs, with a combined pool of up to 192GB VRAM, enabling us to compute brute force searches of $k = 5$ in approximately 7 minutes.

3.5.2 PCA Analysis

To find an approximate high-performing set of k filters without brute force search, we use a greedy selection scheme closely related to Matching Pursuit [25]. The method uses a low-dimensional subspace induced by the target functions and iteratively selects filters whose projections onto this subspace are large and as mutually “orthogonal” as possible.

Let $\mathbf{X} \in \mathbb{R}^{3 \times d}$ denote the discretized target CMFs over ω_d (with $d = |\omega_d|$) and let $\mathbf{F} \in \mathbb{R}^{N \times d}$ denote the candidate filter-sensor rows, i.e. row i is $f_i[\lambda] \sigma[\lambda]$. We form a truncated basis for the target subspace via an SVD (singular value decomposition) of $\mathbf{X}^\top$ and then greedily select k rows of $\mathbf{F}$ using orthogonalized residual norms.

Algorithm 2 PCA / Matching-Pursuit filter selection and weight estimation

Require: Target matrix $\mathbf{X} \in \mathbb{R}^{3 \times d}$; candidate rows $\mathbf{F} \in \mathbb{R}^{N \times d}$; number of selected filters k .

Ensure: Selected indices $I \subset \{1, \dots, N\}$ with $|I| = k$ and weights $\mathbf{M} \in \mathbb{R}^{3 \times k}$.

- 1: Compute thin SVD: $\mathbf{X}^\top = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^\top$.
 - 2: Set $r \leftarrow \min(3, k)$ and $\mathbf{U}_r \leftarrow \mathbf{U}(:, 1:r)$.
 - 3: Project candidates into target subspace: $\mathbf{G} \leftarrow \mathbf{F} \mathbf{U}_r$ $\triangleright \mathbf{G} \in \mathbb{R}^{N \times r}$
 - 4: Initialize $I \leftarrow \emptyset$ and orthonormal basis $\mathbf{Q} \leftarrow [\]$.
 - 5: **for** $t = 1$ to k **do**
 - 6: **if** $t = 1$ **then**
 - 7: $i^* \leftarrow \arg \max_i \|\mathbf{G}_{i,:}\|_2$.
 - 8: $\mathbf{q} \leftarrow \mathbf{G}_{i^*,:} / \|\mathbf{G}_{i^*,:}\|_2$.
 - 9: **else**
 - 10: Compute projection onto current span: $\hat{\mathbf{G}} \leftarrow (\mathbf{G} \mathbf{Q}) \mathbf{Q}^\top$.
 - 11: Orthogonal component: $\mathbf{G}_\perp \leftarrow \mathbf{G} - \hat{\mathbf{G}}$.
 - 12: Set scores $s_i \leftarrow \|\mathbf{G}_{\perp,i,:}\|_2$ and force $s_i = -\infty$ for $i \in I$.
 - 13: $i^* \leftarrow \arg \max_i s_i$.
 - 14: $\mathbf{q} \leftarrow \mathbf{G}_{\perp,i^*,:} / \|\mathbf{G}_{\perp,i^*,:}\|_2$.
 - 15: **end if**
 - 16: $I \leftarrow I \cup \{i^*\}$; $\mathbf{Q} \leftarrow [\mathbf{Q} \ \mathbf{q}]$.
 - 17: **end for**
 - 18: Form selected row-matrix $\mathbf{F}_I \in \mathbb{R}^{k \times d}$ by stacking rows $i \in I$.
 - 19: Least-squares weights (MSE): $\mathbf{M} \leftarrow \mathbf{X} \mathbf{F}_I^+$.
 - 20: **Optional:** if using a non-MSE loss L_d , refine $\mathbf{M}$ via gradient descent initialized at $\mathbf{X} \mathbf{F}_I^+$.
-

In our implementation, the optional refinement step is applied when the loss is not purely MSE (e.g., SMAPE or noise-aware variants): we treat $\mathbf{M}$ as a learnable parameter, initialize it from the pseudo-inverse solution, and take a small number of gradient descent steps to reduce L_d . To this end, we use the Adam algorithm as implemented in `torch.optim.Adam`.

To find the optimal filter combination without brute force search, we can use PCA analysis to find approximate solutions.

This method initially creates one $3 \times |N|$ matrix $\mathbf{M}$ including all N filters, and iteratively selects until reaching k filters and thus our final $3 \times k$ matrix of weights. Specifically, the selection is performed by projecting $\mathbf{F}$ onto the space spanned by the target $\mathbf{X}$ matrix, finding the filter

with the highest norm following the Matching Pursuit method outlined in Mallat and Zhang [25]. We iteratively apply this methodology k times, each time on a subset of the resultant orthonormal basis formed by subtracting the selected filter’s projection.

This method is comparatively fast, able to compute the $k = 5$ case in 19 seconds on a single L4 GPU. This method also scales well, with $k = 8$ runtime at only 20 seconds.

The relative timings for Brute Force explode whereas heuristic algorithms (PCA and LASSO) are nearly constant time in k . A complete performance analysis is conducted in 4.3.

3.5.3 PCA Analysis with Macbeth Target

For $k > 3$, the original PCA Matching-Pursuit procedure can become weakly constrained because the target space induced directly by $\mathbf{X} \in \mathbb{R}^{3 \times d}$ is only three-dimensional. To provide a richer objective, we instead construct a higher-dimensional target subspace using the Macbeth / X-Rite ColorChecker patch reflectances. Intuitively, this biases filter selection toward spanning the diversity of common surface reflectances, which often improves performance at larger k .

Let $\mathbf{R} \in \mathbb{R}^{P \times d}$ be a matrix of P Macbeth patch reflectance spectra sampled on the same grid ω_d (with $P = 24$ for a standard chart). We compute a thin SVD of $\mathbf{R}^\top$ and keep the top $r = \min(P, k)$ components to obtain a basis $\mathbf{U}_r \in \mathbb{R}^{d \times r}$. As before, we project candidate filters into this basis and greedily select indices by maximizing the norm of the component orthogonal to the span of previously selected projections.

Algorithm 3 PCA / Matching-Pursuit selection using Macbeth patch subspace

Require: Macbeth reflectance matrix $\mathbf{R} \in \mathbb{R}^{P \times d}$; candidate rows $\mathbf{F} \in \mathbb{R}^{N \times d}$; desired number of filters k ; target CMFs $\mathbf{X} \in \mathbb{R}^{3 \times d}$.

Ensure: Selected indices $I \subset \{1, \dots, N\}$ with $|I| = k$ and weights $\mathbf{M} \in \mathbb{R}^{3 \times k}$.

```

1: Compute thin SVD of Macbeth basis:  $\mathbf{R}^\top = \mathbf{U}\Sigma\mathbf{V}^\top$ .
2: Set  $r \leftarrow \min(P, k)$  and  $\mathbf{U}_r \leftarrow \mathbf{U}(:, 1:r)$ .
3: Project candidates:  $\mathbf{G} \leftarrow \mathbf{F}\mathbf{U}_r$   $\triangleright \mathbf{G} \in \mathbb{R}^{N \times r}$ 
4: Initialize  $I \leftarrow \emptyset$  and orthonormal basis  $\mathbf{Q} \leftarrow [\ ]$ .
5: for  $t = 1$  to  $k$  do
6:   if  $t = 1$  then
7:      $i^* \leftarrow \arg \max_i \|\mathbf{G}_{i,:}\|_2$ .
8:      $\mathbf{q} \leftarrow \mathbf{G}_{i^*,:} / \|\mathbf{G}_{i^*,:}\|_2$ .
9:   else
10:     $\hat{\mathbf{G}} \leftarrow (\mathbf{G}\mathbf{Q})\mathbf{Q}^\top$ .
11:     $\mathbf{G}_\perp \leftarrow \mathbf{G} - \hat{\mathbf{G}}$ .
12:     $s_i \leftarrow \|\mathbf{G}_{\perp,i,:}\|_2$  and force  $s_i = -\infty$  for  $i \in I$ .
13:     $i^* \leftarrow \arg \max_i s_i$ .
14:     $\mathbf{q} \leftarrow \mathbf{G}_{\perp,i^*,:} / \|\mathbf{G}_{\perp,i^*,:}\|_2$ .
15:   end if
16:    $I \leftarrow I \cup \{i^*\}$ ;  $\mathbf{Q} \leftarrow [\mathbf{Q} \ \mathbf{q}]$ .
17: end for
18: Form selected matrix  $\mathbf{F}_I \in \mathbb{R}^{k \times d}$  by stacking rows  $i \in I$ .
19: Fit weights to the original CMFs (MSE):  $\mathbf{M} \leftarrow \mathbf{X}\mathbf{F}_I^+$ .
20: Optional: refine  $\mathbf{M}$  with gradient descent for non-MSE losses.
```

In the original PCA Matching Pursuit as outlined above, we have an underdefined problem for $k > 3$, as our 3 dimensional target space may be nearly spanned by the projection of the first 3 filters. One way to mitigate this issue is to project our space onto that as defined by the Macbeth Color Checker patches. Using this methodology, we define a higher-dimensional space and thus a lower incremental value per added filter, with a better result at high k . However, we

will observe that this technique only accurately reproduces human eye color matching functions such as the CIE XYZ family of functions, and fails to generalize to other applications. That being understood, this method is able to achieve better metrics at high k . For practical purposes, the difference is immaterial due to the cost of increasing k above 10 or so, in terms of size of components and the capture time increase inherent to adding additional filters.

3.5.4 LASSO Selection

To obtain a sparse (small- k) filter set without explicitly searching combinations, we also evaluate a LASSO-style selection procedure. Here we optimize a full weight matrix over the entire candidate set and use an $\ell_{2,1}$ -type regularizer (group sparsity) to drive many filter-columns to near-zero magnitude.

Let $\mathbf{F} \in \mathbb{R}^{K \times d}$ denote the candidate filter-sensor rows and let $\mathbf{X} \in \mathbb{R}^{3 \times d}$ denote the discretized target CMFs. We solve for a dense weight matrix $\mathbf{M} \in \mathbb{R}^{3 \times K}$ by minimizing

$$\min_{\mathbf{M}} L_d(\mathbf{M}\mathbf{F}, \mathbf{X}) + \lambda \sum_{j=1}^K \|\mathbf{m}_{:,j}\|_2, \quad (3.8)$$

where $\mathbf{m}_{:,j}$ is the j th column of $\mathbf{M}$. The second term promotes column-wise sparsity: when a column norm becomes small, the corresponding filter contributes negligibly to reconstruction.

After optimization, we rank filters by their column norms $\|\mathbf{m}_{:,j}\|_2$ and keep the top k filters. Finally, we refit weights on the selected subset via a pseudo-inverse (least squares) to obtain the final $\mathbf{M} \in \mathbb{R}^{3 \times k}$ used for evaluation.

Algorithm 4 LASSO / group-sparse filter selection

Require: Target $\mathbf{X} \in \mathbb{R}^{3 \times d}$; candidates $\mathbf{F} \in \mathbb{R}^{K \times d}$; desired number of filters k ; regularization weight λ .

Ensure: Selected indices I and final weights $\mathbf{M}_I \in \mathbb{R}^{3 \times k}$.

- 1: Initialize $\mathbf{M} \leftarrow \mathbf{X}\mathbf{F}^+$ ▷ Pseudo-inverse warm start
 - 2: **for** $t = 1$ to T **do**
 - 3: $\hat{\mathbf{X}} \leftarrow \mathbf{M}\mathbf{F}$
 - 4: $\mathcal{L} \leftarrow L_d(\hat{\mathbf{X}}, \mathbf{X}) + \lambda \sum_{j=1}^K \|\mathbf{m}_{:,j}\|_2$
 - 5: Update $\mathbf{M}$ by one Adam step on $\mathcal{L}$
 - 6: **end for**
 - 7: Compute column norms $n_j \leftarrow \|\mathbf{m}_{:,j}\|_2$ for $j = 1, \dots, K$
 - 8: Select $I \leftarrow$ indices of the top- k values of n_j
 - 9: Form $\mathbf{F}_I \in \mathbb{R}^{k \times d}$ and refit $\mathbf{M}_I \leftarrow \mathbf{X}\mathbf{F}_I^+$
-

3.5.5 Brute Force Search Augmented PCA

While brute force methods guarantee minimum possible error by investigating the entire search space, large k implementations render this technique impractical. Our PCA method outlined above cannot always identify the optimal filter set, and has a nonnegligible MSE shortfall vs. exhaustive search for all $k > 1$.

To address this problem, we use a blended method which finds a PCA solution for k_1 filters, and then considers all further possible combinations from a further k_2 filters, for a total $k = k_1 + k_2$. This provides a middle ground to the performance-accuracy tradeoff. There is negligible MSE difference between brute force and PCA methods at $k_2 > 1$ for all k_1 . Further, for $k_2 < 4$, the added time from the exhaustive search step is between 5-20 seconds on the 8 aforementioned L4 GPUs.

We note here that the MSE shortfall observed above has mostly disappeared, with PCA having a significant performance improvement over Brute Force. Further results are considered in 4.3 and a full performance overview is included in the Appendix (4.3.2).

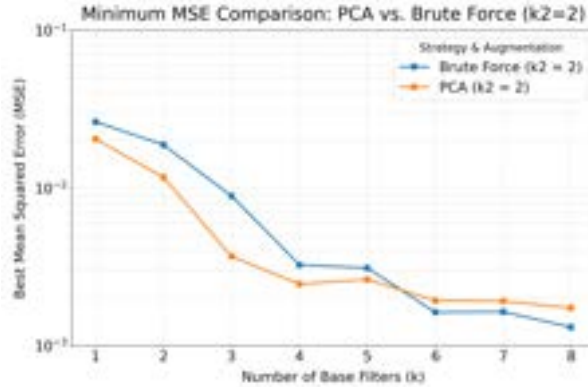


Figure 3.8: MSE comparison between PCA and brute force selection methods ($k_2 = 2$)

One challenging problem introduced by the extra choice of k_2 is an explosion of state space and a violation of certain earlier assumptions regarding brute force superiority. Whereas in the original problem, we have a monotonically decreasing relationship between MSE and k in the brute force case (if the 'best' filter would decrease MSE, a weight of 0 would be applied), the introduction of the brute force rounds requires us to intelligently choose k_2 . Then, under practical considerations in a filter wheel of limited k , the choice of k_1 vs. k_2 becomes a difficult to define and hard to optimize tradeoff. In fact, certain artifacts emerge in the comparison between brute force and heuristic algorithms with $k_2 > 1$, such as improved MSE performance for the PCA algorithm at $k = 2$ as shown above.

Presumably a brute force algorithm would necessarily find the global minimum MSE, however, with $k_2 > 0$ and the brute force solution 'branching' (in essence, considering k_2 filters from the k_1 brute force filters) may miss the optimal filter combination. Therefore, any brute force solution with $k_2 > 0$ should be considered with caution. There is value in augmenting a brute force solution, as runtime is greatly reduced, but we find that the effect of mixing search algorithms at an intermediate value provides an efficient yet accurate middle ground for practical values of k .

3.6 Computation

For our simulated camera, we use the Python `color` and `colour-science` libraries, with the CIE 1931 or 2015 2 Degree Standard Observer respectively, depending on whether we use the CIE 1931 or CIE 2006 XYZ functions. We use a D65 illuminant and ColorChecker N Ohta reflectance patches. XYZ responses were converted to CIELAB using the `color.xyz2lab` function to convert XYZ signals to lab before ΔE_{00} computation.

The specific algorithm is below, including the result for the best `top_m` filter combinations, for N patches in a color checker:

Algorithm 5 Evaluation Pipeline: ΔE_{00} Computation

Require: Original values $\mathbf{Y} \in \mathbb{R}^{N \times d}$; Reconstructed values $\hat{\mathbf{Y}} \in \mathbb{R}^{N \times d}$; white point $\mathbf{W}_{\text{ref}}$.

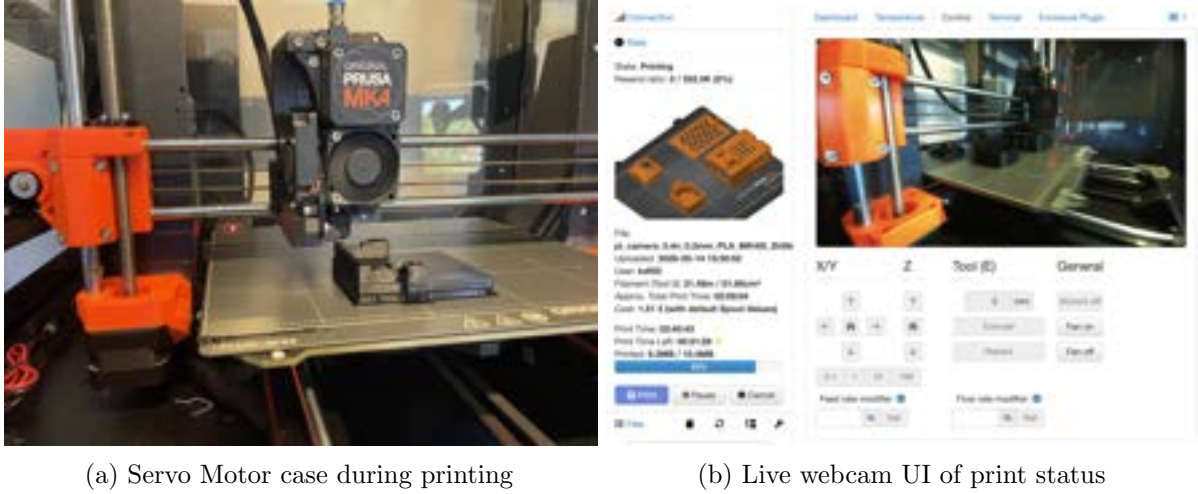
Ensure: Average Mean Squared Error (MSE_{avg}) and Average CIEDE2000 ($\Delta E_{00,\text{avg}}$).

- 1: Initialize sum of MSE $E_{\text{MSE}} \leftarrow 0$.
 - 2: Initialize sum of color differences $E_{\Delta 00} \leftarrow 0$.
 - 3: **for** $i = 1$ to N **do**
 - 4: Convert $\mathbf{Y}_{i,:}$ to CIELAB color space: (L_i, a_i, b_i) using $\mathbf{W}_{\text{ref}}$.
 - 5: Convert $\hat{\mathbf{Y}}_{i,:}$ to CIELAB color space: $(\hat{L}_i, \hat{a}_i, \hat{b}_i)$ using $\mathbf{W}_{\text{ref}}$.
 - 6: Compute perceptual differences: $\Delta L', \Delta C', \Delta H'$ from CIELAB pairs.
 - 7: Calculate weighting functions S_L, S_C, S_H and rotation term R_T .
 - 8: Compute sample CIEDE2000 distance:
$$\Delta E_{00}^{(i)} \leftarrow \sqrt{\left(\frac{\Delta L'}{k_L S_L}\right)^2 + \left(\frac{\Delta C'}{k_C S_C}\right)^2 + \left(\frac{\Delta H'}{k_H S_H}\right)^2} + R_T \left(\frac{\Delta C'}{k_C S_C}\right) \left(\frac{\Delta H'}{k_H S_H}\right)$$
 - 9: $E_{\Delta 00} \leftarrow E_{\Delta 00} + \Delta E_{00}^{(i)}$.
 - 10: **end for**
 - 11: $\Delta E_{00,\text{avg}} \leftarrow E_{\Delta 00}/N$.
 - 12: **return** $\Delta E_{00,\text{avg}}$
-

3.7 Physical Camera

3.7.1 3D Printed Housing

We construct a 3D printed housing to arrange the camera components such that the Raspberry Pi may control the servo motor as well as the camera capture. Designs for individual components such as Raspberry Pi and Camera housings are available freely at Printables.com [58, 59, 60], with recommended print settings. The Prusa MK4S 3D printer [61, 62, 63] uses fused filament fabrication to construct high-quality plastic structures.

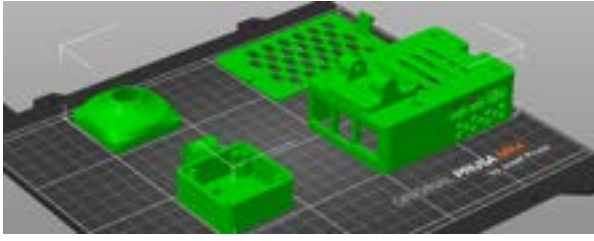


(a) Servo Motor case during printing

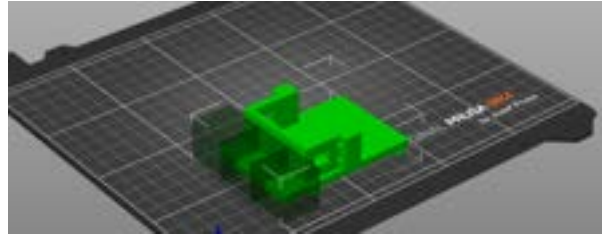
(b) Live webcam UI of print status

Figure 3.9: 3D Printed Case

PrusaSlicer also provides the ability to merge components, for example the camera stand onto the Raspberry Pi case or the Servo motor holder onto a Raspberry Pi baseplate. The software proceeds to optimize the print for structural stability while minimizing filament.



(a) Raspberry Pi and Camera Housing



(b) Servo Motor Housing

Figure 3.10: 3D Print Model (Screenshots of PrusaSlicer software)

The final product contains a central housing to contain the Raspberry Pi microcontroller, with a camera stand atop facing the servo motor with attached filter wheel. When capturing photos, we ensure stable lighting and subject immobility while placing the camera on a flat surface. Capture is controlled by a computer connected to the same WiFi network as the Raspberry Pi, wherein one can send commands to the device for photo capture and servo motor adjustment via Raspberry Pi Connect app.

Below, we show the camera with a plain background. Note the removable camera mount, enabling filter wheel replacement for increasing or decreasing k and implementation of different Color Matching Function reconstructions.



Figure 3.11: Printed Camera and Servo Motor Setup

The total cost of construction of the entire camera is £118.24, not including the recycled PLA used in 3D printing, nor the small number of fasteners used to hold various components in place. Specifically, the RaspberryPi 5 can be found for £65.50 on PiHut, OV9281 Monochrome

Camera for £28.80, the Rosco Supergel swatch book for £13.80, and the MG996R 180° servo motor for £13.14 [64, 65, 66, 67]. For comparison, consumer cameras range from £120-£250 or higher [68]. A further study of consumer-facing cameras is included in section 6.1.4.

Chapter 4

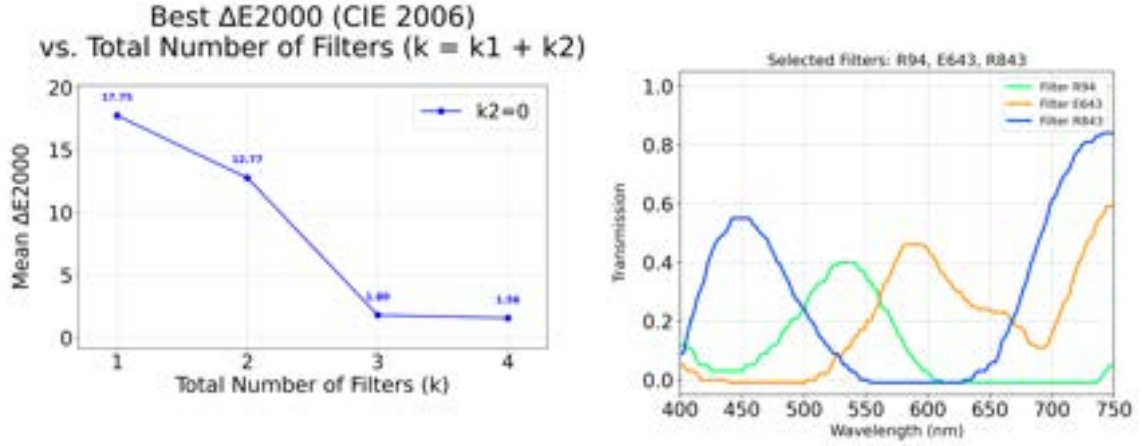
Evaluation

We evaluate our simulated camera stacks by comparing the reconstructed XYZ color matching functions from our selected filters to the target XYZ functions, summarized by the MSE loss. We also consider the color difference ΔE_{00} for a set of reflectances. This latter method effectively approximates the human eye’s perception of color difference, and is therefore well suited for a practical evaluation of our reconstruction [69]. For the latter evaluation, we use the Macbeth Color Checker [49] as our test set, and render XYZs and our RGBs (found from our method to best approximate XYZs) under a D65 illuminant.

Then, we test the prescribed filters on our physical camera, deployed in a controlled lighting laboratory setting.

4.1 MyColor Filter Set

First, we review the results of the simulated camera on a set of 494 filters drawn from the Rosco MyColor tool. We approximate the transmission function of each filter by scraping the web tool for the spec sheet and approximating the relative location of the line on the chart. Therefore, we use this set as an illustrative example in a fully simulated environment, having significant flexibility of filter selection. Also, we cannot effectively benchmark our filter selection process past $k = 4$ due to the computational complexity of selecting $k > 4$ filters, with already 2.5×10^9 combinations for the $k = 4$ case. However, we notice that even with just $k = 3$ filters in use from the MyColor dataset, we are able to effectively attain a simulated ΔE_{00} of 1.80, already nearly ‘just noticeable difference’ [52] across the mean of the 24 color patches. Of course, for a three dimensional target function, an improvement in reconstruction quality over the first 3 filters is fairly trivial, but the speed at which we can attain high quality reconstructions is impressive. We use the brute force method of filter selection to attain these results.

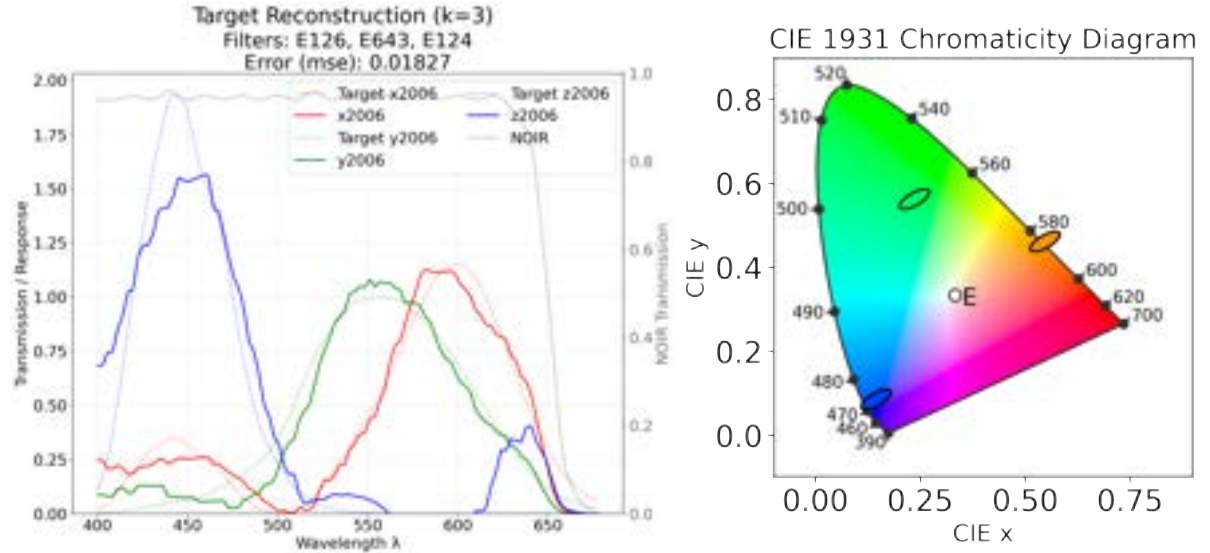


(a) Mean ΔE_{00} of best combination for each number of (b) Transmission functions for optimal MyColor scraped filters $k < 5$ $k = 3$ solution

Figure 4.1: MyColor $k < 5$ performance and selected filters.

Indeed, we can visually inspect the transmission functions above to see that our functions share approximate modes with the XYZ functions, given that the right tails will be neutralized above 650nm by the presence of our IR-cutoff filter. We also note some artifacts due to the scanning of specification sheets, and some resultant irregularities in the reconstruction.

Their transformations after multiplication by $\mathbf{M}$ are shown below, including a simulated IR-cutoff filter which cuts off most transmission above 650 nm. These are overlaid on the CIE 2006 XYZ (from LMS) functions.



(a) Transformed transmission functions with CIE 2006 (b) Best filters from MyColor at $k = 3$ on the XYZ spectral response functions 1931 color gamut

Figure 4.2: MyColor $k = 3$ filters in spectral and chromaticity space.

The filters' chromaticities can also be viewed overlaid onto the CIE 1931 Chromaticity Diagram in CIE x and CIE y space to see that these filters 'span' the entire gamut, and hence a linear combination of these filters may effectively reproduce any color on the gamut. While

we primarily use the New CIE 2006 LMS functions and their respective XYZ transformations (denoted CIE 2006), a significant body of research exists referring to the 1931 XYZ functions which provide helpful benchmarks for our performance. Further, the CIE 1931 Color Gamut diagram provides a natural and familiar demonstration of color within the CIE x and CIE y coordinates.

More rigorous analysis requires the use of ΔE_{00} calculations using a color checker, and in this case, our mean ΔE_{00} across 24 patches reached the value of 1.80.

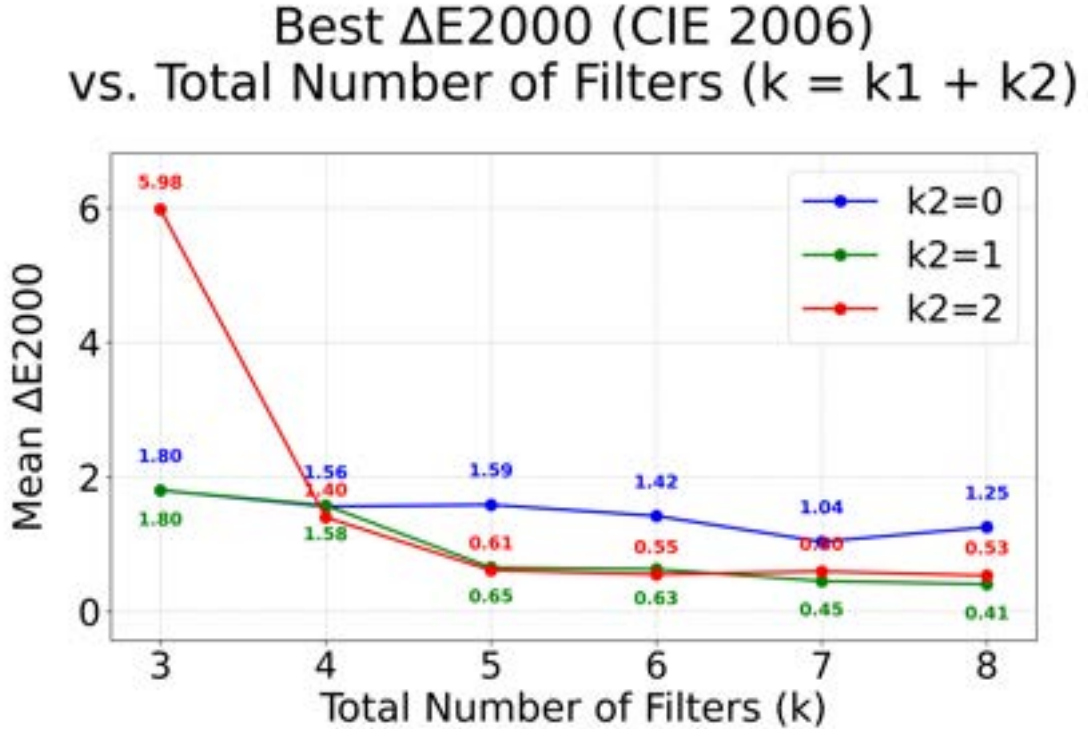


Figure 4.3: Scraped Filter Set ΔE_{00} Progression for $k \in \{3...8\}$ and $k_1 \in \{0, 1, 2\}$

While ΔE_{00} of 1.80 is sufficient for human-centered applications in controlled lighting, we can improve upon this result and strive for lower MSE results. With the full set of 494 filters, we are unable to compute a brute-force solution in reasonable time, and must use heuristic solutions. Fortunately, the heuristic solutions augmented with a few brute force rounds, such as those enumerated in section 3.5.2 and 3.5.5 provide effective solutions. We show up to $k = 8$ filters' ΔE_{00} progressions as a function of k above.

This result supports our intuition that we can achieve ΔE_{00} values comparable with state of the art results using our analysis. Importantly, the improvement in ΔE_{00} largely flattens after $k = 5$, providing useful knowledge as to the necessary number of filters to attain a high-quality reconstruction. We include a rendering from the CAVE dataset reconstructed with the $k = 5 (k_1 = 4, k_2 = 1)$ solution for an example of a practical simulated filter wheel setup. We also provide the $k = 2 (k_1 = 2, k_2 = 0)$, $\Delta E_{00} = 12.77$ solution for comparison.

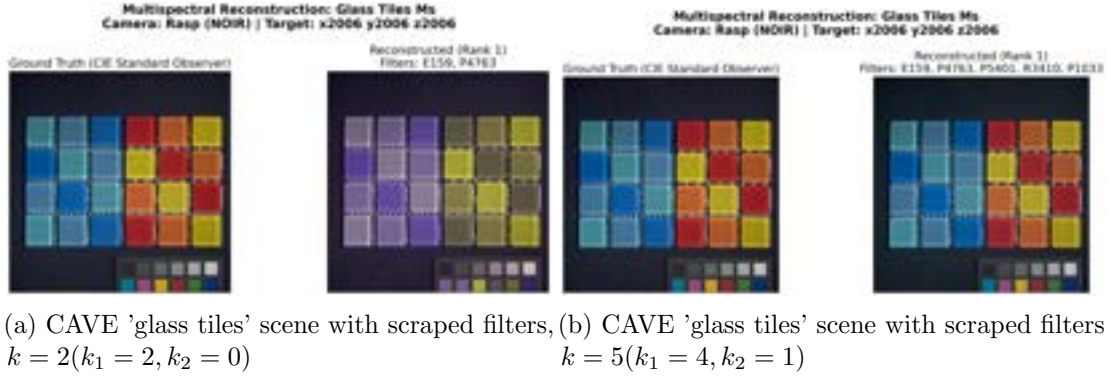


Figure 4.4: Comparison of different CAVE renders with scraped filters

We note that the set of 494 filters provides a large candidate pool, making it comparatively easy to reconstruct the desired target functions, subject to our ability to correctly identify a good combination. In the next section, we explore a similar analysis on a smaller set of filters, searching for an equivalent reconstruction quality and more easily calculating the performance difference against the best possible solution as found by brute force, which we cannot feasibly compute for the $N = 494$ sized set of filters.

4.2 Rosco Supergel Set in Simulated Camera

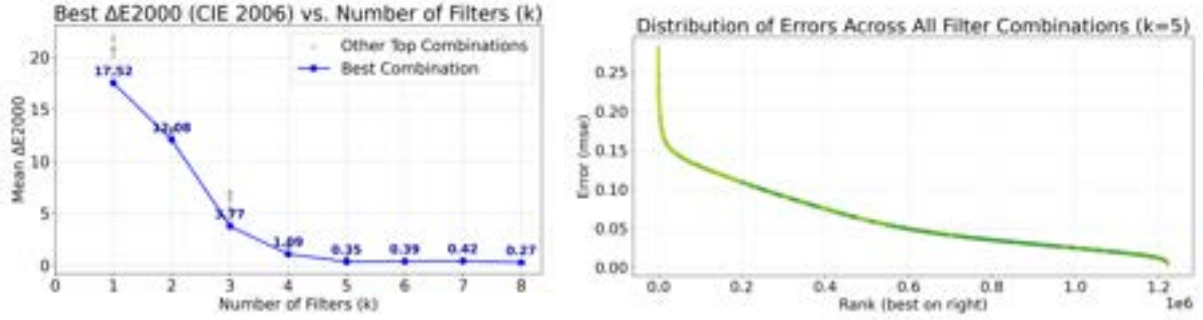
We now inspect our physical filter set. These filters will be those used in the 3D printed camera, and this section will inform our selection for the physical camera. Therefore, we provide more rigorous analysis of the optimal choice at each set.

In this section, we progressively increase the number of filters k and analyze the reconstruction quality for each. We comment on the increases in reconstruction quality afforded by each incremental extra capture, understanding that each filter increases capture time and possibility of a 'bad capture.' Specifically, because we require a mechanical servo turn between each capture, if an object in our scene moves, or worse, if our camera moves during capture, we must use capture alignment algorithms for image processing. While these algorithms may become a valuable component of the camera pipeline, they are beyond the scope of this work.

To maintain consistency across this demonstration, we run all the simulations in the following section with an IR-cutoff filter included. Additionally, for the computations of ΔE_{00} , we use the Python `color` and `colour-science` libraries, with the CIE 1931 or 2015 2 Degree Standard Observer respectively, depending on whether we use the CIE 1931 or CIE 2006 XYZ functions. D65 illuminant from the same library is used as well as ColorChecker N Ohta reflectance patches. XYZ responses were converted to CIELAB using the `color.xyz2lab` function to convert XYZ signals to lab before ΔE_{00} computation. We compute the white point as the sum of the Y channel divided by D65 light, following the `color` package. A Von Kries white balance is also applied following West and Brill [70].

4.2.1 MSE, ΔE_{00} and CAVE Results for $k \in \{3, 4, 5\}$

We will provide some filter wheel prescriptions along the various values of k we consider for our filter wheel. In Figure 4.5, we show the best ΔE_{00} of our approximation at each k filter subset of the Rosco Supergel set, where $k \in \{1, \dots, 8\}$. For completeness and surety of optimal combination identification, we use brute force search in this section, thus $k = k_1$ and we omit k_2 . In this section, we continue to use the IR-cutoff filter and a standard MSE loss for consistency versus the prior section and other works.



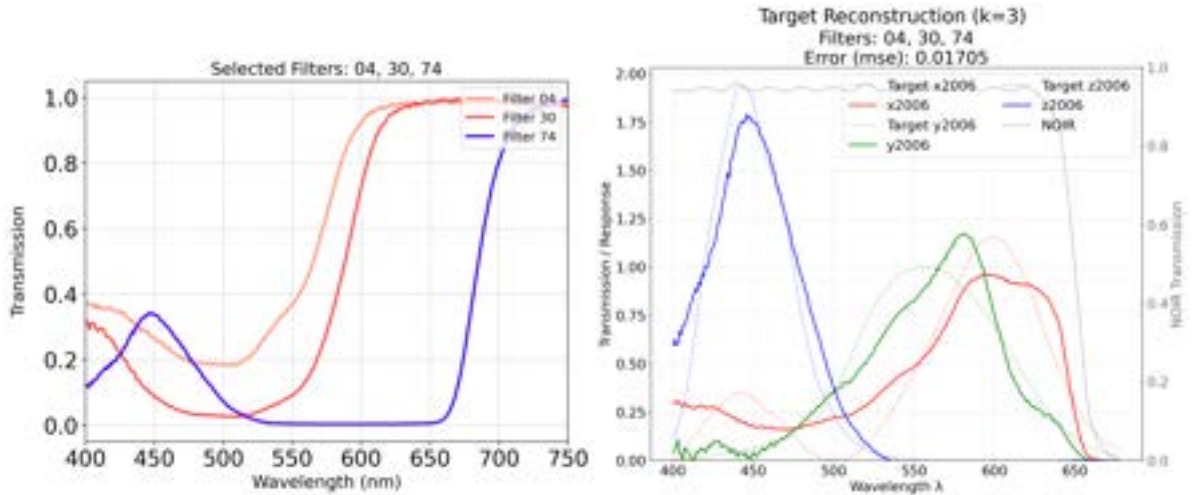
(a) Best ΔE_{00} for each number of filters k

(b) Error distribution for $k = 5$ solutions

Figure 4.5: Example simulated reconstruction and overall Supergel set performance.

4.2.1.1 Solution for $k = 3$

The best filter combination for $k = 3$, clearly a poor spectral fit, see Fig. 4.7, achieves a tolerable mean ΔE_{00} of 3.77. This is in the just noticeable difference threshold of 2-4 [52], indicating that the reconstructed colors may be distinguishable from the target colors under certain conditions. The 24 patches below show the original in the top left of each pane and the simulated reconstruction in the bottom right. It is evident there is a visible color difference.



(a) Transmission Functions

(b) Best XYZ reconstruction

Figure 4.6: Supergel results for $k = 3$.

We also include some characteristics of the filters themselves, specifically the 1931 gamut locations and their measured transmission functions. For instance, 4.6a shows the transmission functions of the selected filters for the $k = 3$ solution, and 4.7b shows the perceived location of the filter on the color gamut.

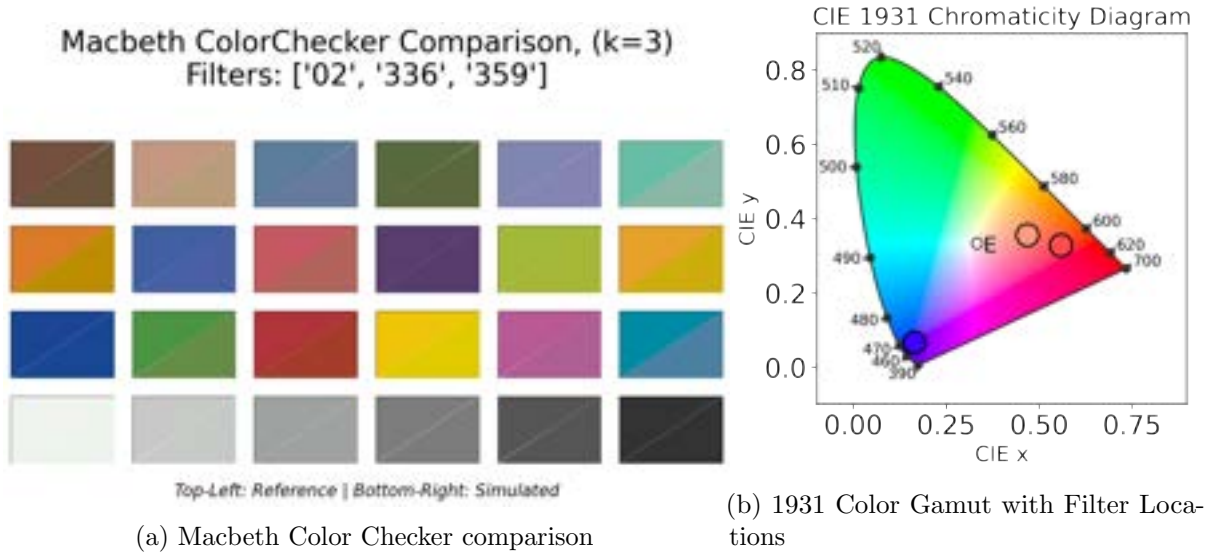


Figure 4.7: Supergel results for $k = 3$.

The CAVE dataset renders are included at each k for a qualitative comparison of reconstruction quality.



Figure 4.8: $k = 3$ CAVE Multispectral Dataset Renders

4.2.1.2 Solution for $k = 4$

We see a noticeable improvement in MSE and in ΔE_{00} with $k = 4$, measuring 1.09, slightly above the 'just noticeable difference' of 1.0 [52]. The CAVE dataset reconstruction approaches the ground truth reconstruction, although the human eye may observe slight differences in the overall image.

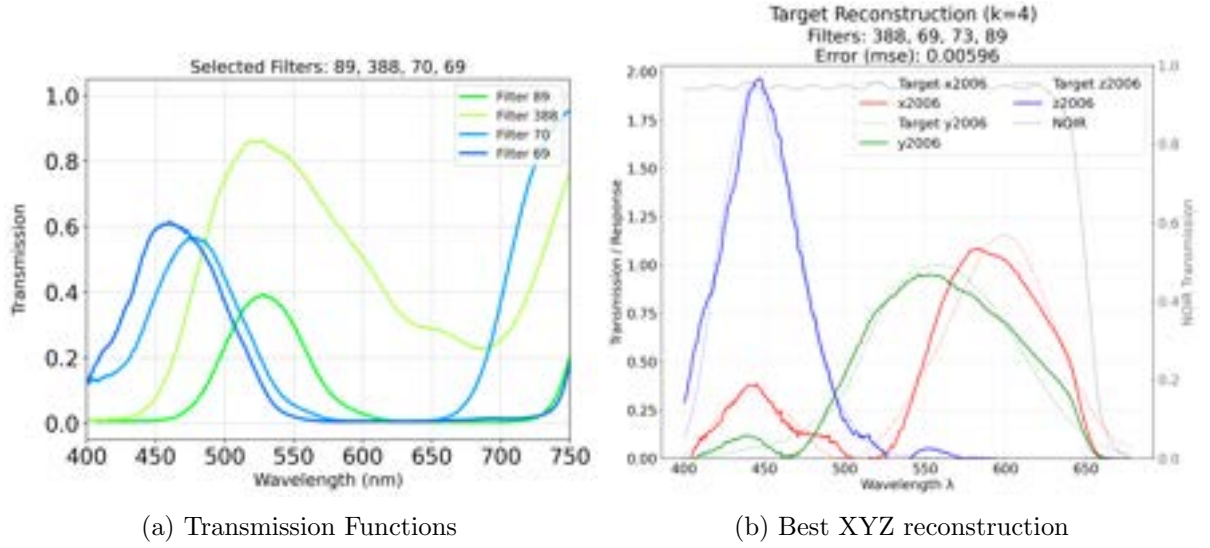


Figure 4.9: Supergel results for $k = 4$.

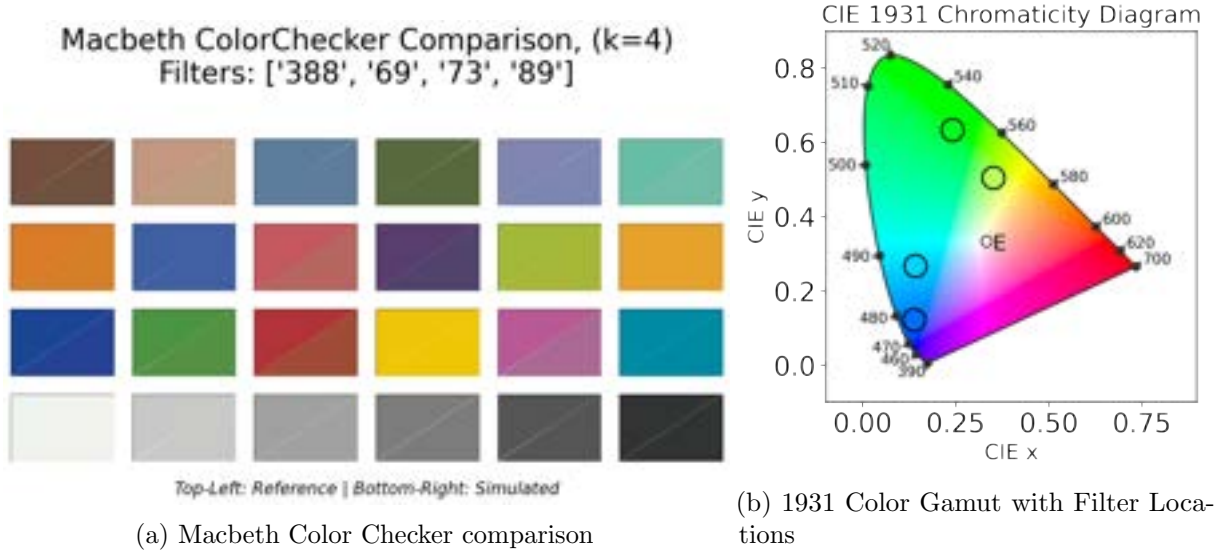


Figure 4.10: Supergel results for $k = 4$.

In comparison to the $k = 3$ case, we see relatively little transmission above the 650nm range, yet a reconstruction may struggle to recreate red hues and the 15th Macbeth patch, the dark red patch, has a ΔE_{00} of about 9, significantly worse than the rest. Conceptually, we can see that the gamut has little coverage over the red corner of the 1931 color gamut.

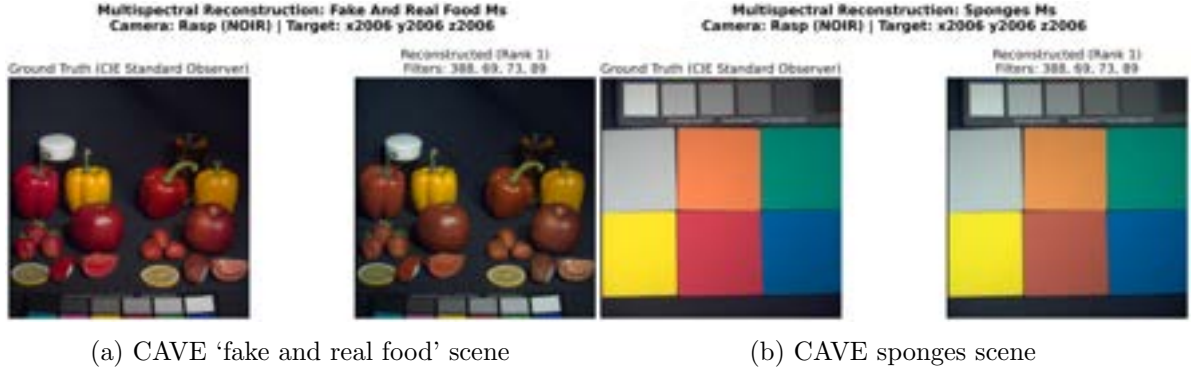


Figure 4.11: $k = 4$ CAVE Multispectral Dataset Renders

4.2.1.3 Solution for $k = 5$

For the $k = 5$ case, the ΔE_{00} reaches 0.35 and clearly improves on the reconstruction of the target curves. Importantly, the worst individual ΔE_{00} recorded amongst the Macbeth patches are patches 2 and 15, the tan and red patches. The CAVE reconstruction is quite hard to distinguish from the ground truth.

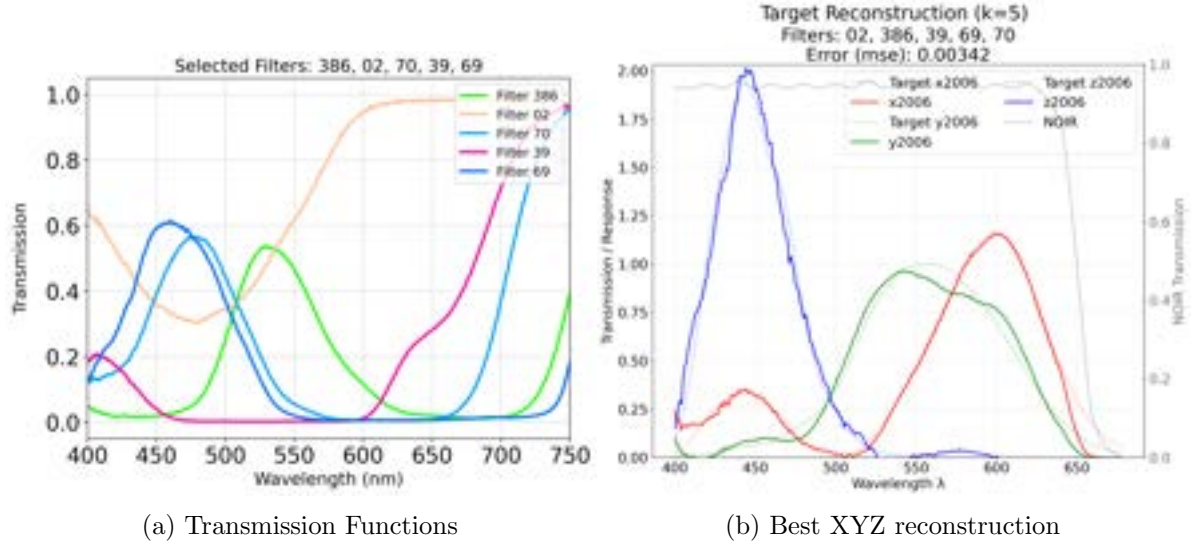


Figure 4.12: Supergel results for $k = 5$.

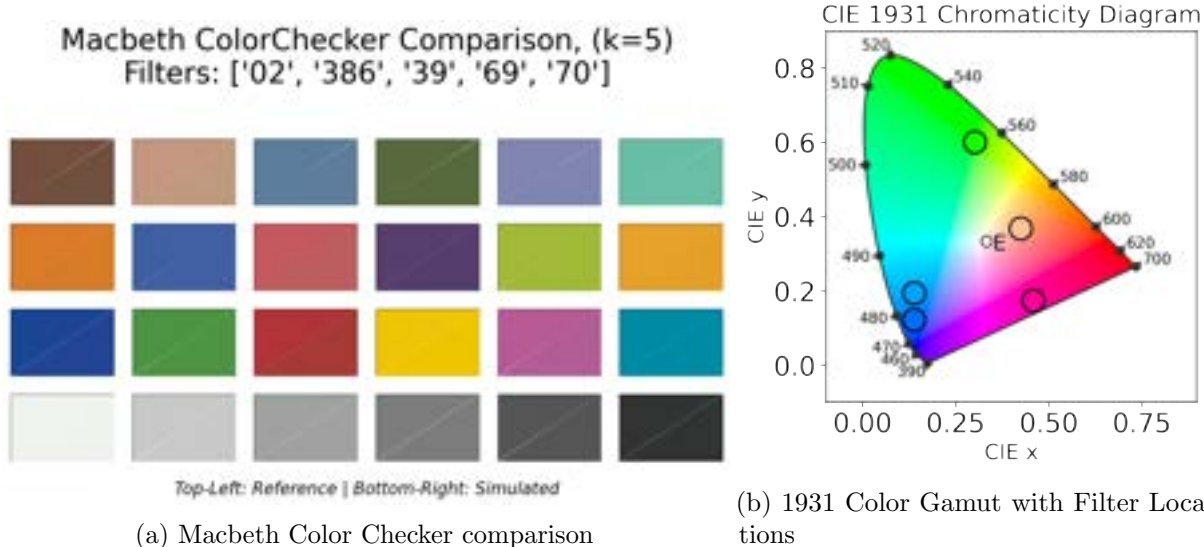


Figure 4.13: Supergel results for $k = 5$.



Figure 4.14: $k = 5$ CAVE Multispectral Dataset Renders

Additional captures from the CAVE dataset are included in the appendix.

4.2.2 CIE 2006 10-degree CMF

We focus our prior research on the 2-degree CMF derived by Stockman & Sharpe [71], but also show that with minimal overhead, we can provide an effective filter selection for the 10 degree color matching functions enumerated in the same body of research. In this context, the 'degree' refers to the field of view of the observer. While we only achieve $\Delta E_{00} < 1$ at $k = 6$, this is a reasonable result. We note that the original PCA algorithm performs worse than the PCA2 algorithm (Algorithm 3), the former failing to find a selection which identifies a combination with $\Delta E_{00} < 1$.

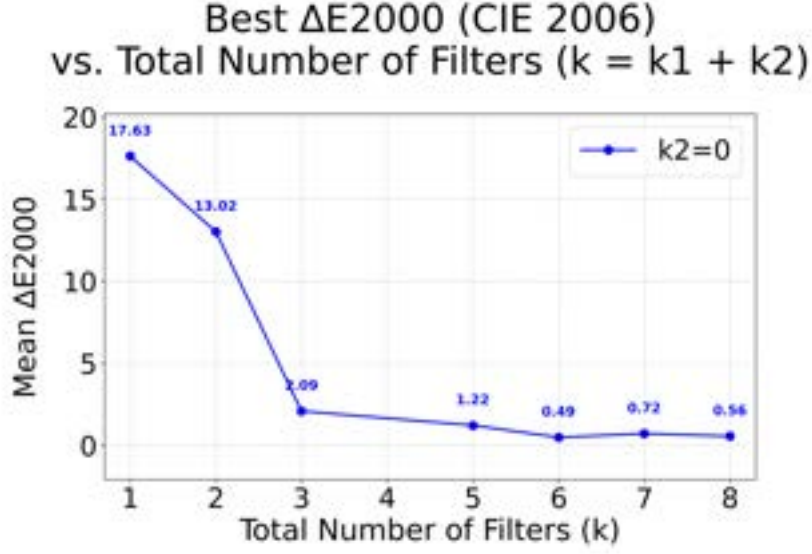


Figure 4.15: ΔE_{00} for $k \in \{1, 2 \dots 8\}$ for the 10 degree observer

We present the spectral reconstruction of the 10 degree function, noting the increased MSE. The 10 degree function, even at the best $k = 6$ solution, does have some notable and visible differences in the spectral reconstruction. Hence, the ΔE_{00} progression has a worse minimum, only achieving $\Delta E_{00} < 1$ at $k = 6$.

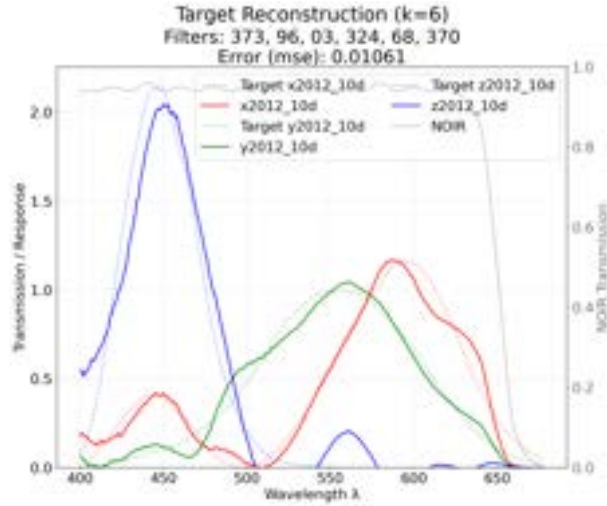


Figure 4.16: Best Spectral Reconstruction for $k = 6$

4.3 Algorithm Comparison

We provide a brief analysis of our various filter selection methodologies. Note that for the Brute Force methodology, we omit $k_1 > 5$ as this was not executed on our hardware in reasonable time. Additional charts are available in section 4.3.2.

4.3.1 Performance Comparison

In this section, we analyze the best MSE and ΔE_{00} for the brute force and PCA algorithms, in the simulated Rosco Supergel set. We see that counterintuitively, the best ΔE_{00} performances are found at the $k_1 = 3$ and $k_1 = 4$, $k_2 = 2$ filter selectors. The performance is also not monotonically decreasing due to the difficulties of the filter selection problem, but these ΔE_{00} metrics are below the just noticeable difference of 1, and the simulated reconstructions are essentially indistinguishable from ground truth.

k	k_1	k_2	MSE				Mean ΔE_{00}			
			PCA	PCA2	LASSO	Brute	PCA	PCA2	LASSO	Brute
3	3	0	0.0468	0.0521	0.0366	0.0170	2.6571	2.2899	4.1681	2.9162
4	3	1	0.0338	0.0326	0.0300	0.0085	1.5895	1.9938	3.7888	0.8602
	4	0	0.0390	0.0334	0.0303	0.0060	2.2856	1.7851	1.0793	2.0186
5	3	2	0.0183	0.0179	0.0245	0.0048	0.5799	0.9080	0.7568	0.3844
	4	1	0.0330	0.0273	0.0238	0.0041	1.7279	0.8761	0.7449	0.4481
	5	0	0.0252	0.0273	0.0163	0.0027	1.0123	1.2558	0.5738	1.0192
6	4	2	0.0176	0.0260	0.0067	0.0032	0.6566	0.8714	0.4519	0.4168
	5	1	0.0172	0.0231	0.0114	0.0019	0.6552	0.9072	0.4980	0.5272
	6	0	0.0181	0.0099	0.0161	–	0.6129	0.5415	0.5589	–
7	5	2	0.0150	0.0192	0.0067	0.0014	0.4709	0.6962	0.4505	0.3870
	6	1	0.0163	0.0060	0.0112	–	0.5020	0.4363	0.4714	–
	7	0	0.0172	0.0121	0.0112	–	0.5965	0.6928	0.4714	–
8	6	2	0.0139	0.0047	0.0064	–	0.4467	0.6310	0.4176	–
	7	1	0.0158	0.0101	0.0063	–	0.4547	0.7027	0.5595	–
	8	0	0.0141	0.0041	0.0077	–	0.4925	0.5521	0.3753	–

We chart the performance of each algorithm in the context of shortfall vs. the brute force algorithm. Note that we do not run brute force for $k > 5$, and therefore this data is missing for those points.

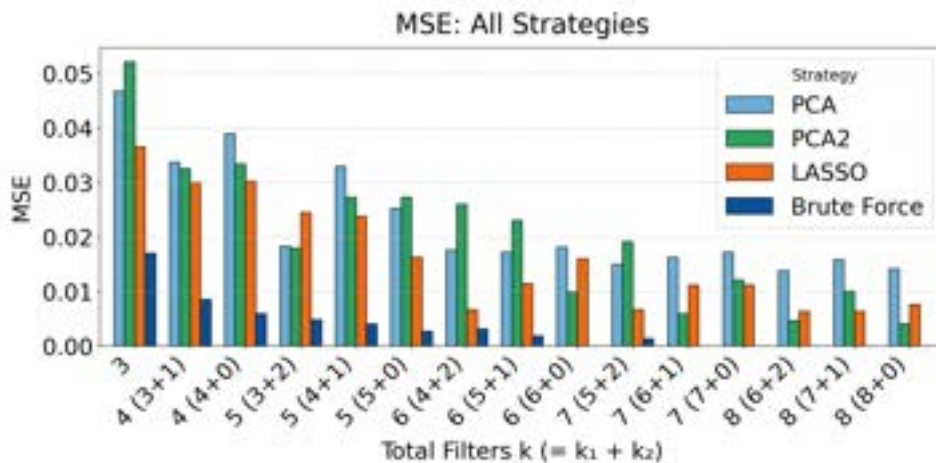


Figure 4.17: MSE Comparison Between Strategies

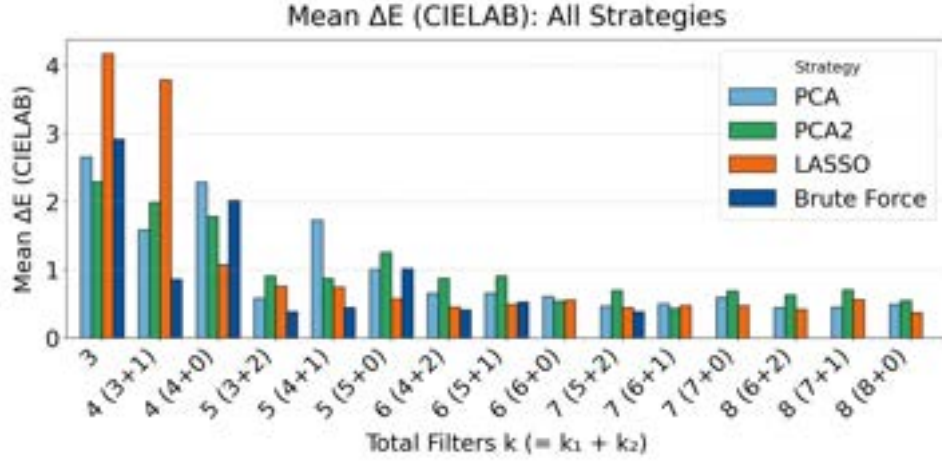


Figure 4.18: ΔE_{00} Comparison Between Strategies

4.3.2 Timing Comparison

We provide the performance for each algorithm run on the hardware specified in section 3.5.1. Note that the heuristic algorithms (PCA, PCA2, LASSO) each only use one GPU, although their augmentation algorithms are deployed across all 8 GPUs. We see that the brute force algorithm performance degrades significantly with increasing k_1 , whereas the heuristic algorithms remain mostly constant.

			Base Time (s)				Total Time (s)			
k	k_1	k_2	PCA	PCA2	LASSO	Brute Force	PCA	PCA2	LASSO	Brute Force
3	3	0	6	6	8	31	9	9	10	34
4	3	1	6	6	8	31	26	27	28	52
	4	0	6	6	8	315	9	9	11	319
5	3	2	6	6	8	31	27	26	29	52
	4	1	6	6	8	315	27	26	29	336
	5	0	6	6	8	6394	9	9	11	6397
6	4	2	6	6	8	315	27	27	29	336
	5	1	6	6	8	6394	27	26	29	6414
	6	0	6	6	8	—	9	9	11	—
7	5	2	6	6	8	6394	27	26	29	6414
	6	1	6	6	8	—	27	27	28	—
	7	0	6	6	8	—	9	9	11	—
8	6	2	6	6	8	—	27	27	30	—
	7	1	6	6	8	—	27	28	29	—
	8	0	6	6	8	—	9	9	11	—

We include some charts relating to the timing records of our filter selection.

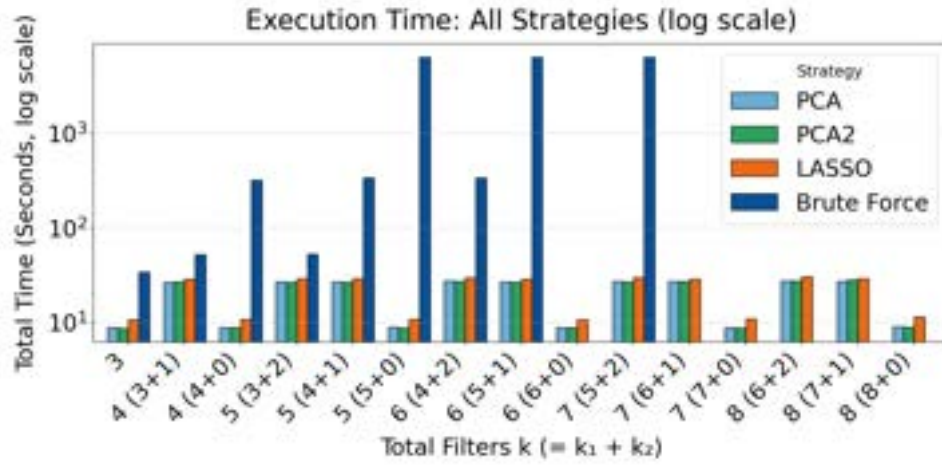


Figure 4.19: All Algorithm Choices Timing Log Scale

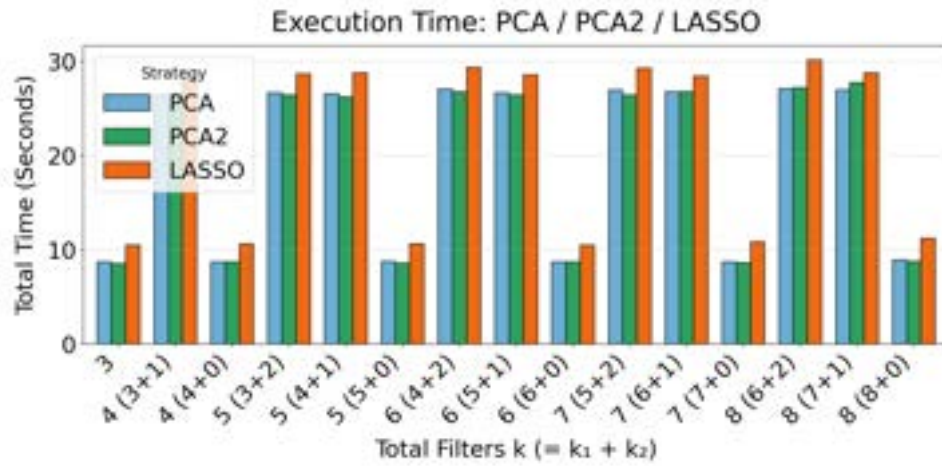
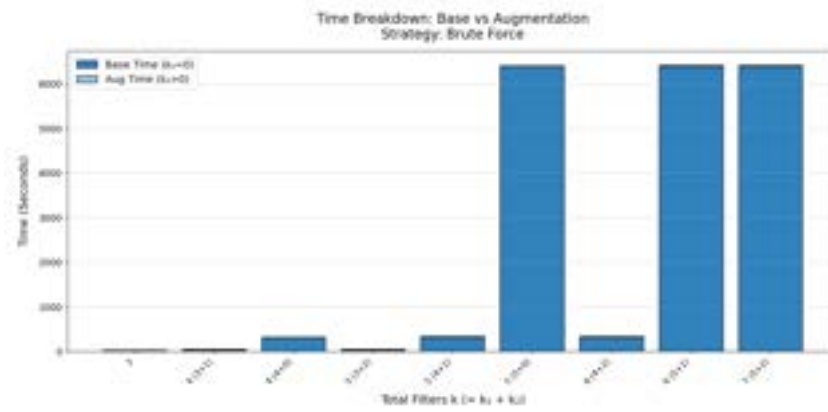
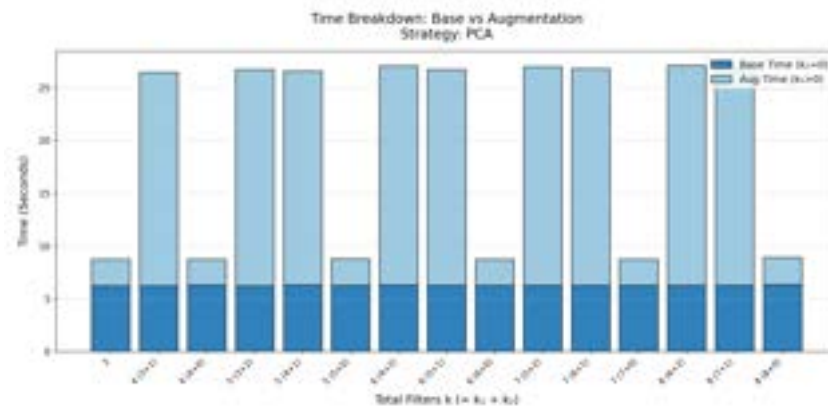


Figure 4.20: All Algorithm Choices Timing, Except Brute Force, Normal Scale

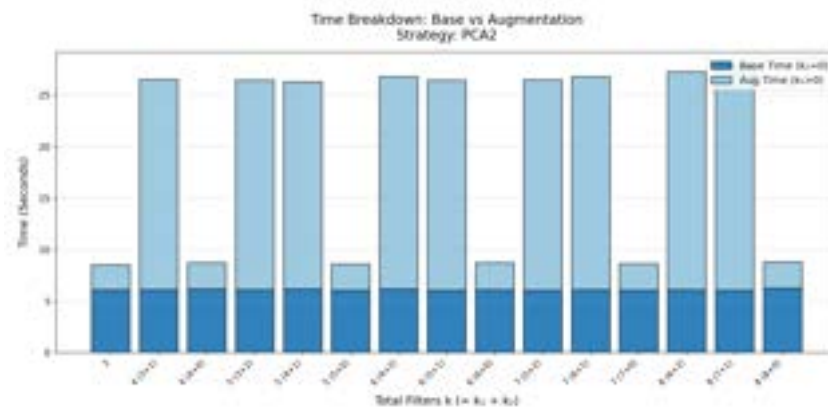
We also break the analysis down by strategy. We can clearly see that most of the heuristic filter selection algorithms run in nearly constant time, with augmented round processing time adding a significant performance penalty. However, on total processing time, the heuristic algorithms still significantly outperform the brute force selection.



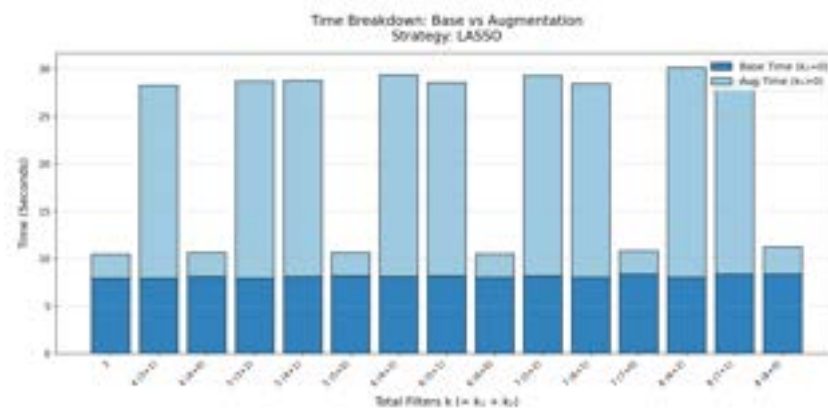
(a) Brute Force Timing



(b) PCA Timing



(c) PCA2 Timing



(d) LASSO Timing

Figure 4.21: Timing Comparison of Various Algorithms

4.3.3 CAVE Dataset

We include images from the CAVE dataset, rendered under various $k \in \{2, 3, 4, 8\}$ values. We consider the set of simulations including an IR-cutoff filter, and filters selected with the brute force methodology, having $k_2 = 0$. Because the $k = 8$ solution takes some time to brute force, we use a cosine similarity function to prune the filters to a set of 45, which reduces reconstruction quality and increases the differences in ΔE_{00} across renders.

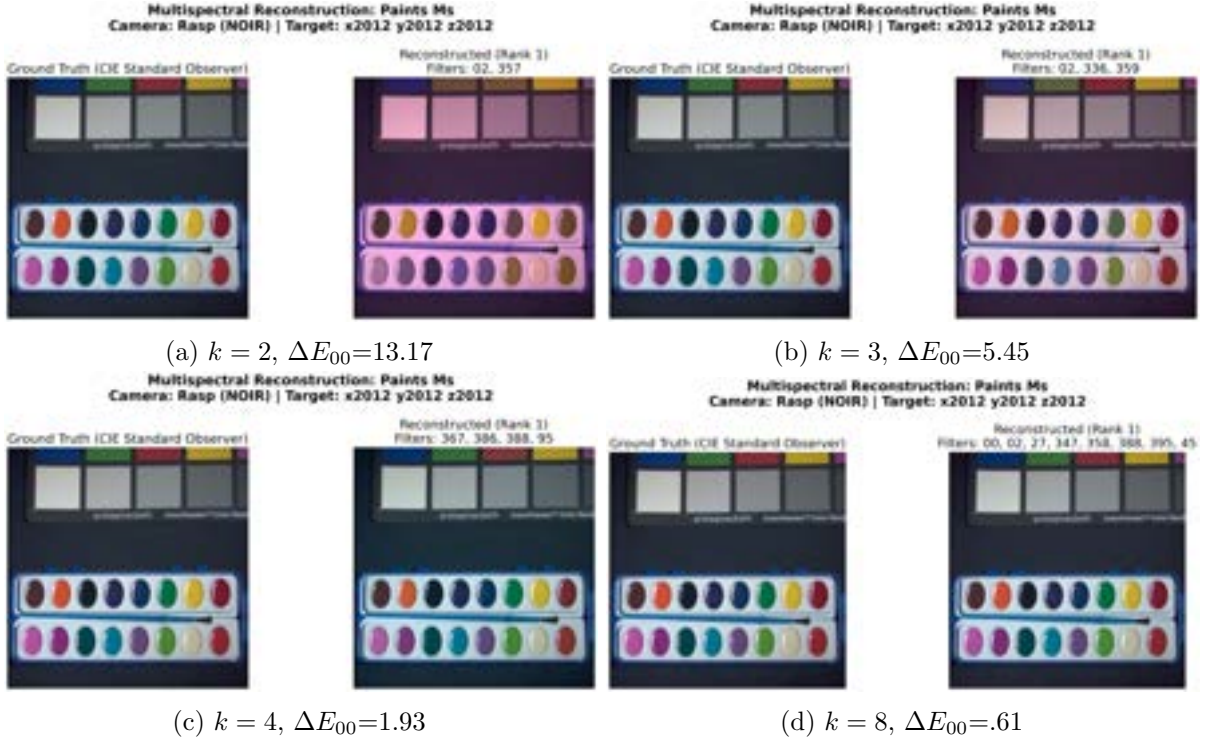


Figure 4.22: CAVE Renders of Paints Multispectral Dataset, With Macbeth ΔE_{00}

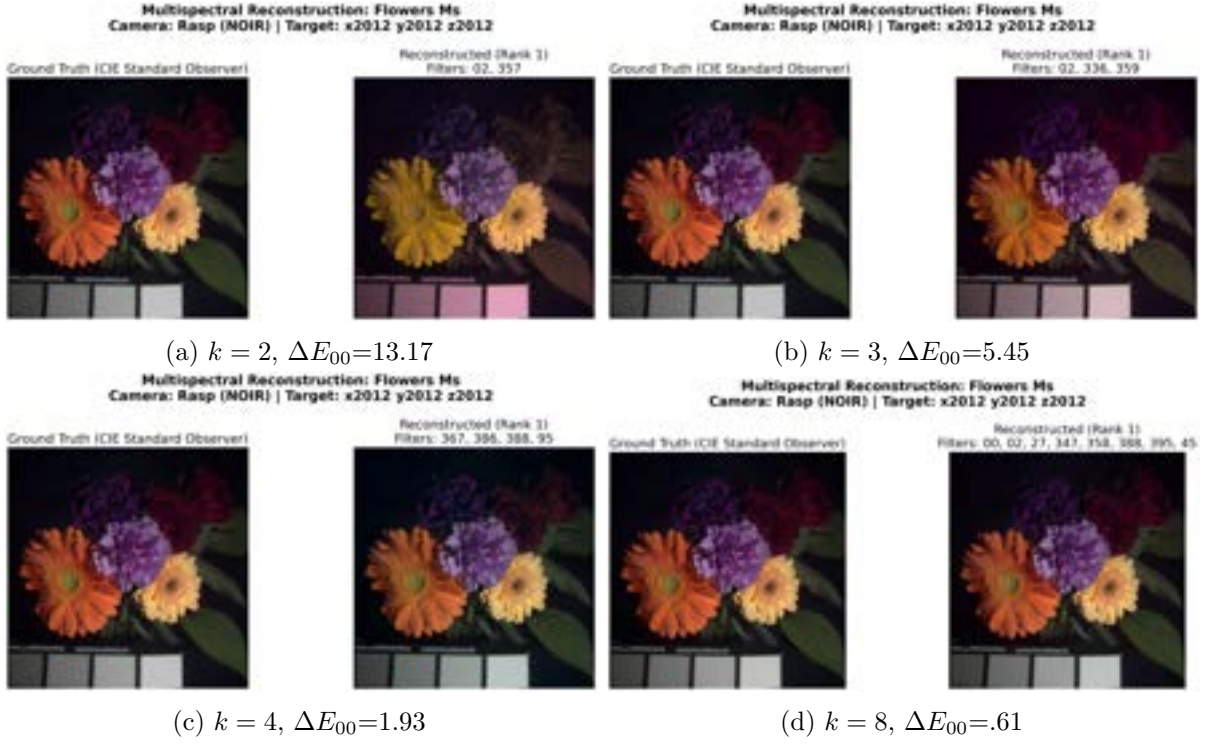


Figure 4.23: CAVE Renders of Flowers Multispectral Dataset, With Macbeth ΔE_{00}

4.3.4 Noise Model Analysis

For analysis of the noisiness of our capture, we switch from the MSE loss function to the MSE'noisy loss function as defined in 1, and we compare the results with increments in the photons parameter. For this analysis, we increment along the k_1 parameter amongst $k_1 \in \{2, 4, 6\}$, fixing $k_2 = 2$. The difference in ΔE_{00} performance using the noise model is significantly larger due to the worse reconstruction quality attained by adding noise to the loss function. Hence, ΔE_{00} attained with $k = 4$ ($k_1 = 2, k_2 = 2$) filters never decreases past 14.

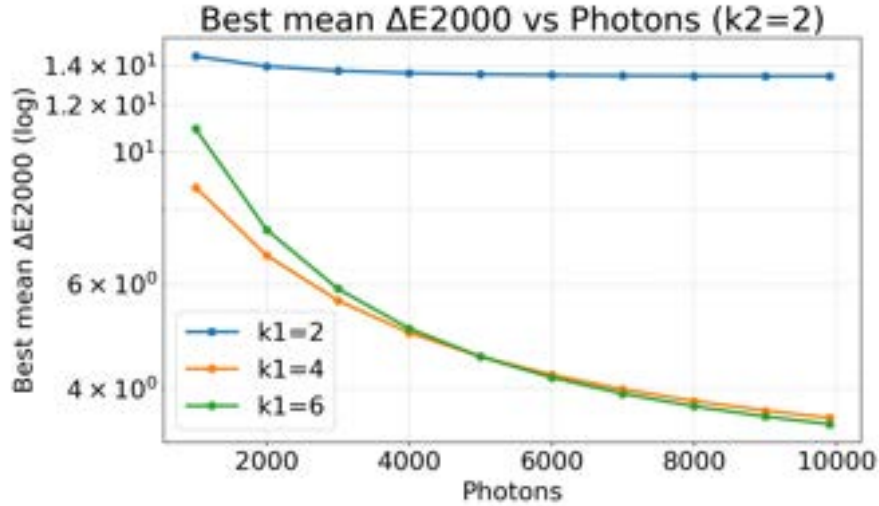


Figure 4.24: Best ΔE_{00} as a function of Photons

The noise model is not directly used in our filter selection process, but informs our understanding of filter wheel dynamics and shutter speed. Because we are mostly concerned with

creating an optimal camera, we capture with sufficiently long exposure time and lighting to ensure that the effect of noise is minimized.

4.4 Physical Camera with Rosco Supergel Set

For the initial evaluation of our physical camera stack, we stage an environment under the same Lilliput [56] illuminant in a lab environment. We position the illuminant about 2 meters above the scene to tune the intensity of light into the camera aperture, and place our camera (removed from the housing) on a tripod, aimed at the stage.



(a) Camera on tripod in lab, side angle



(b) Camera on tripod, rear angle

Figure 4.25: Lab setup for image capture, with deconstructed camera

Then, we test the recommended best performing filter combinations against a multispectral scene including a PMC in a lab environment, using the Rosco Supergel set of filters.

We then take the best prescribed reconstructions for the $k_1 = 4$ and $k_1 = 5$ reconstructions to test our methodology.

Qualitatively, we see that these solutions do not provide perfect reconstructions, and the intensity of some of the hues is dulled. For a more quantitative analysis, we utilize the spectrophotometer in this same lab environment as 3.2 and compute the ground truth x and y values. Then, we can use the same methodology as the simulated pipeline to compute ΔE_{00} .

Here, we have the solution for the $k_1 = 5$ reconstruction.

4.5 Deployed Camera: Lab Results

In this section, we examine the deployed camera. We place the camera on a fixed tripod and capture a scene including a PMCC and some colored items, then attempt to reconstruct the scene from the filtered captures.

4.5.1 Lab Reconstructions

We use the filter selections from our simulation to inform our filter wheel inputs. We place the multispectral scene under an Ianiro Lilliput lamp, [56] with other light sources in the room masked. Captures are taken sequentially, saved to a directory and named with the name of the filter used. Then, reconstruction is done by combination of each capture, multiplied by the combination weights from the simulation and transformed to sRGB.

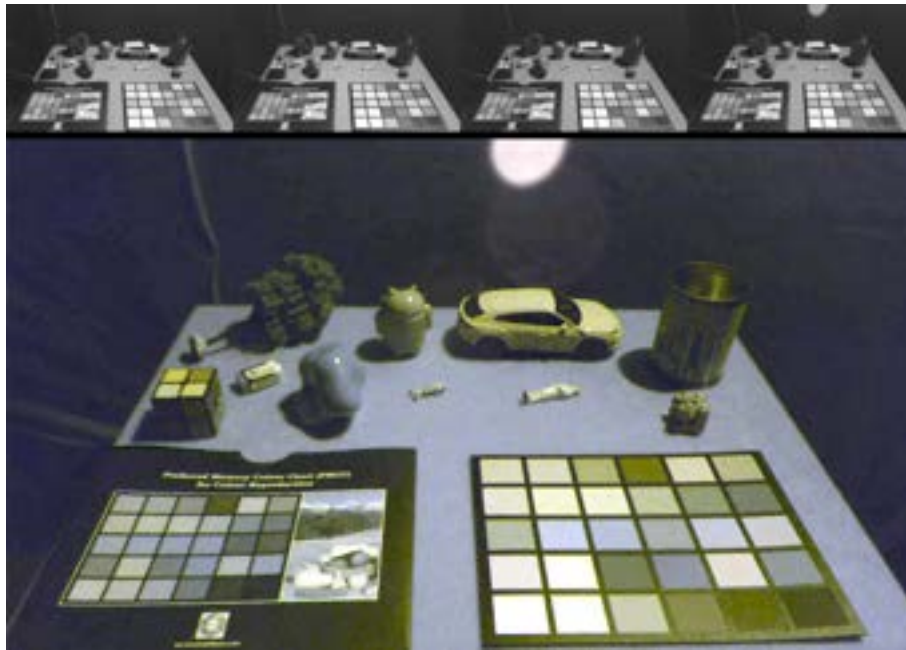
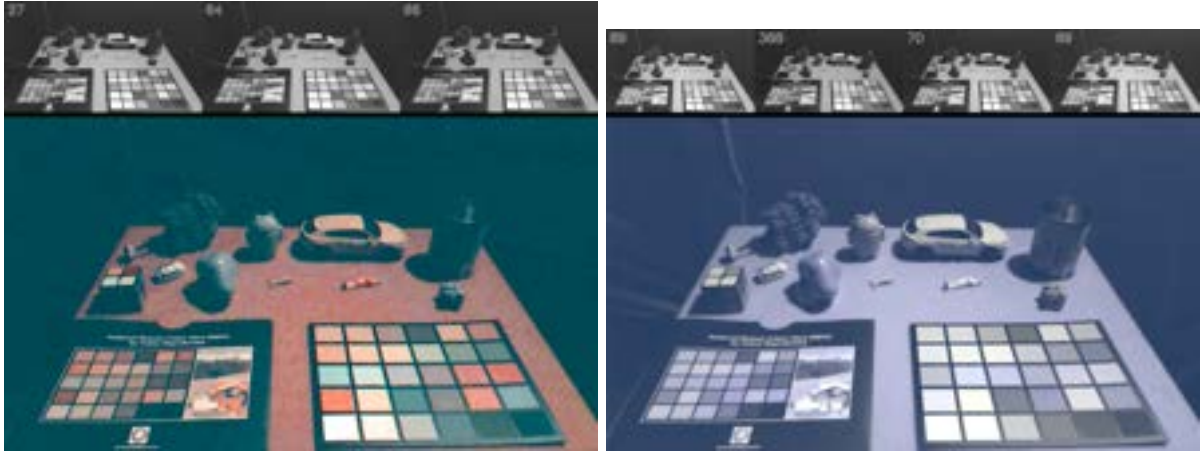


Figure 4.26: $k_1 = 4$ Photo Lab Reconstruction with component captures above



Figure 4.27: $k_1 = 5$ Photo Lab Reconstruction with component captures above

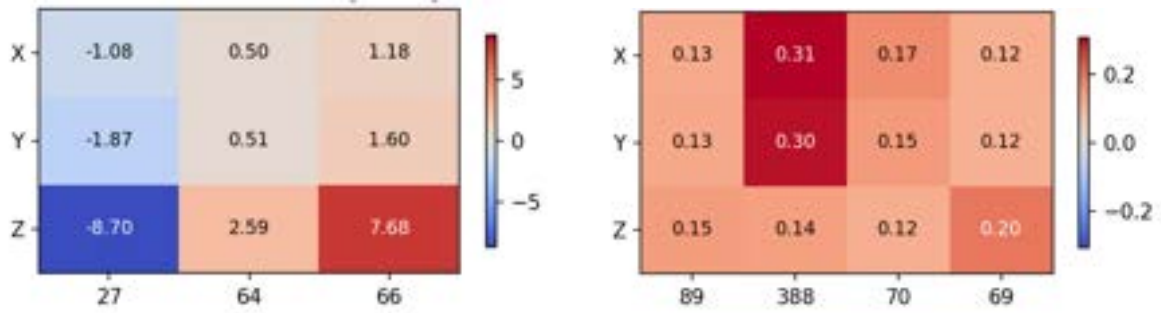


(a) $k = 3$ Lab Reconstruction

(b) $k = 4$ Lab Reconstruction

Figure 4.28: $k = 3$ and $k = 4$ Reconstructions

As per the results in 4.5, we do not expect perfect color reconstructions for the $k < 5$ cases.

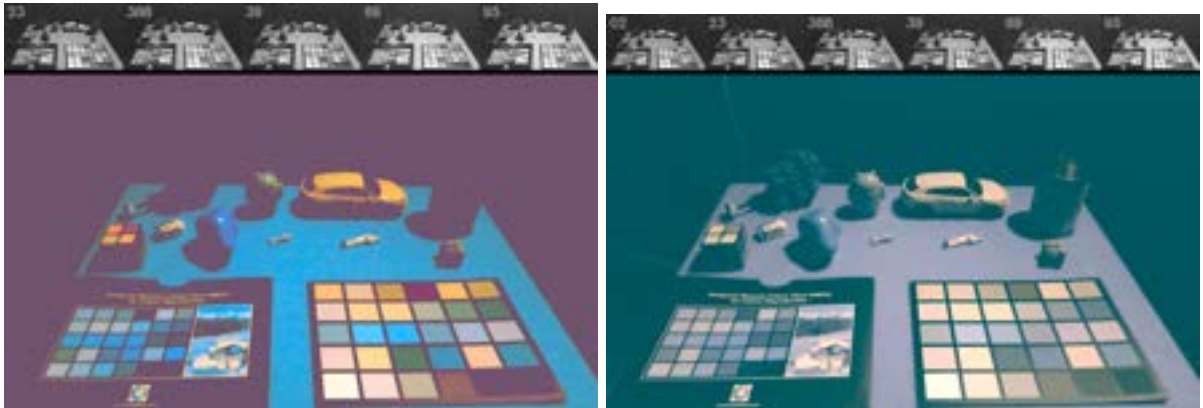


(a) $k = 3$ Combination Weights $\mathbf{M}$ Visualization

(b) $k = 4$ Combination Weights $\mathbf{M}$ Visualization

Figure 4.29: $k = 3$ and $k = 4$ Combination Weight $\mathbf{M}$ Visualizations

Nonetheless, we do exhibit significant tinting of the scenes in all cases, indicating that our combination weights are not accurate for the scene. We measure ΔE_{00} versus ground truth as measured by the spectrophotometer. Unfortunately, we are unable to achieve $\Delta E_{00} < 6$ across any high-performing filter combination from our simulated results.

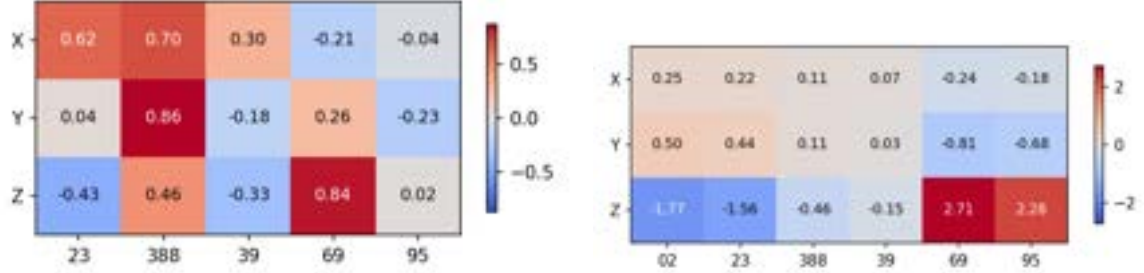


(a) $k = 5$ Lab Reconstruction

(b) $k = 6$ Lab Reconstruction

Figure 4.30: $k = 5$ and $k = 6$ Reconstructions

Further, analyzing the combination weight matrices for the $k = 5$ and $k = 6$ cases, we do not see the artifacts exhibited by high combination weights. Given the consistent tinting of our results, often toward blue colors, we believe that measurement error of the PMCC, or a difference between the laboratory setting versus the simulations, may have an effect in biasing our target values and therefore making our reconstruction poor.



(a) $k = 5$ Combination Weights M Visualization (b) $k = 6$ Combination Weights M Visualization

Figure 4.31: $k = 5$ and $k = 6$ Combination Weight M Visualizations

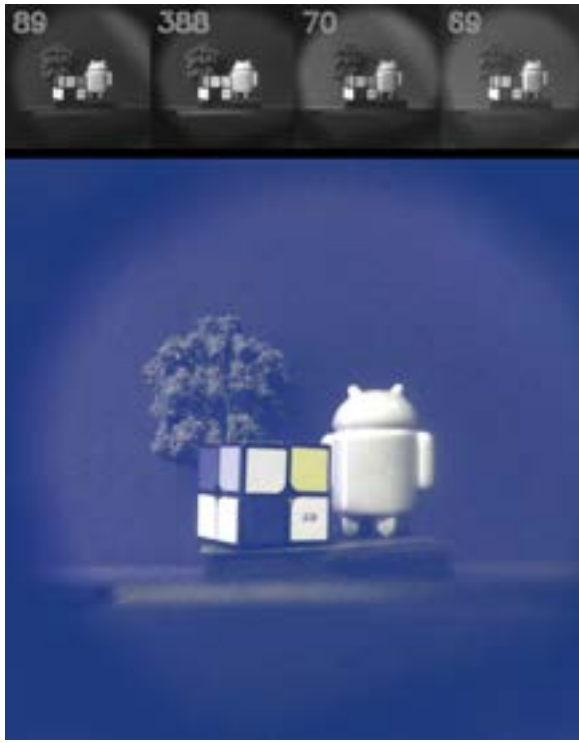
4.6 Deployed Camera: Scene Results

4.6.1 Scene Setup

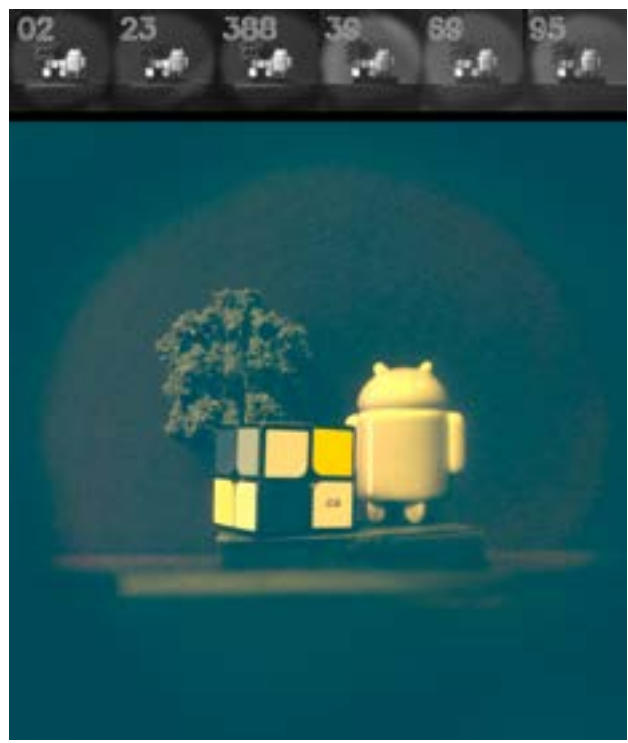


Figure 4.32: Camera and Target

We organize a similar scene for use with the physical camera, having the filter wheel operate between each capture to test the full functionality. We note that the reconstruction quality is even worse than the lab captures, indicating a severe misalignment between the filter selection and reconstruction weights relative to the values required for these conditions.



(a) $k = 4$ scene reconstruction



(b) $k = 6$ scene reconstruction

Figure 4.33: $k = 4$ and $k = 6$ Reconstructions of Rubiks Cube Scene

Chapter 5

Conclusion

We are able to achieve fairly accurate reconstructions of the CIE 1931 and 2006 XYZ color matching functions with as few as $k = 4$ filters, and very high quality reconstructions with $k > 5$ filters. This suggests that a filter wheel system with 6 filters could be sufficient for high-quality color reconstruction. However, the system does have drawbacks, such as shutter speed on the order of seconds and the need for a static scene.

We also find that a hybrid filter selection methodology provides a reasonable middle ground for finding a good combination among a massive search space with a significant cost per combination considered. Especially for non-MSE loss functions, where the computationally more complex gradient descent in addition to the pseudoinverse calculation significantly increases cost per combination, reasonable and efficient filter selection algorithms can provide the difference between an intractable problem, in incorrect filter selection, or a good reconstruction.

We note that the exposure time and lighting conditions of the camera system have significant effects on the reconstruction quality, such that optimal filter choices for different lighting conditions are not necessarily constant in k . The effect of noise on camera capture error, even after taking care to minimize, creates difficulties in reconstruction of captures even in controlled, lab environments. Higher- k filter wheels can also exhibit worse reconstruction in the real world simply due to the challenges inherent to a longer capture time.

Finally, we deploy the camera in real world environments, although show the difficulties inherent in deploying a simulated solution in a physical environment.

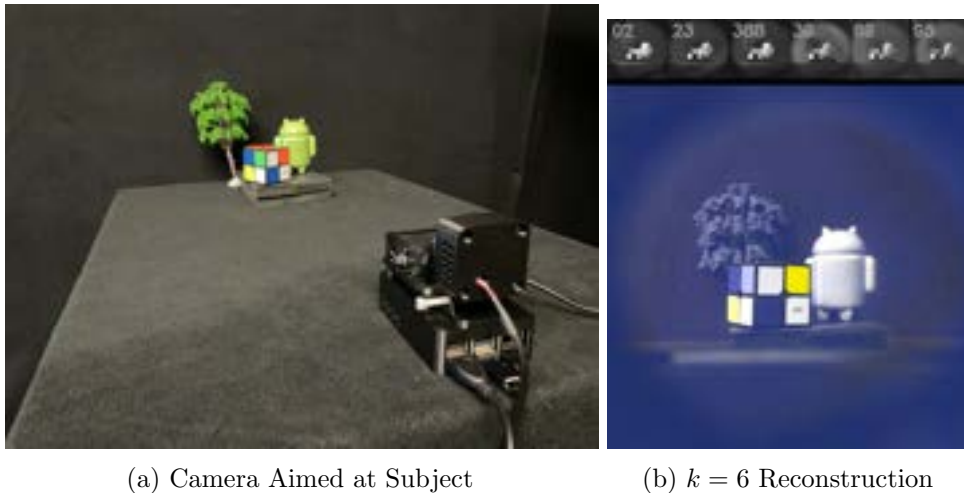


Figure 5.1: Camera with Subject and $k = 6$ Reconstruction

Bibliography

- [1] S. Peyghambari and Y. Zhang, “Hyperspectral remote sensing in lithological mapping, mineral exploration, and environmental geology: an updated review,” *Journal of Applied Remote Sensing*, 2021.
- [2] S. Ortega, H. Fabelo, D. K. Iakovidis, A. Koulaouzidis, and G. M. Callico, “Use of hyperspectral/multispectral imaging in gastroenterology. shedding some-different-light into the dark,” *Journal of Clinical Medicine*, vol. 8, no. 1, p. 36, 2019.
- [3] K. d. C. Mariotti, R. S. Ortiz, and M. F. Ferrão, “Hyperspectral imaging in forensic science: An overview of major application areas,” *Science & Justice*, vol. 63, pp. 387–395, 2023.
- [4] J. Qin, K. Chao, M. S. Kim, R. Lu, and T. F. Burks, “Hyperspectral and multispectral imaging for evaluating food safety and quality,” *Journal of Food Engineering*, vol. 118, pp. 157–171, 2013.
- [5] D. Wu and D.-W. Sun, “Advanced applications of hyperspectral imaging technology for food quality and safety analysis and assessment: A review - part i: Fundamentals,” *Innovative Food Science and Emerging Technologies*, vol. 19, pp. 1–14, 2013.
- [6] K. R. Lee, A. J. Richardson, E. Walowit, M. A. Crognale, and M. A. Webster, “Predicting color matches from luminance matches,” *Journal of the Optical Society of America A (JOSA A)*, vol. 37, no. 4, pp. A35–A43, 2020.
- [7] I.-C. Wu, Y.-C. Chen, R. Karmakar, A. Mukundan, G. Gabriel, C.-C. Wang, and H.-C. Wang, “Advancements in hyperspectral imaging and computer-aided diagnostic methods for the enhanced detection and diagnosis of head and neck cancer,” *Biomedicines*, vol. 12, p. 2315, 2024.
- [8] B. Lu, P. D. Dao, J. Liu, Y. He, and J. Shang, “Recent advances of hyperspectral imaging technology and applications in agriculture,” *Remote Sensing*, 2020.
- [9] D. Scutelnic, R. Muradore, and C. Daffara, “A multispectral camera in the vis–nir equipped with thermal imaging and environmental sensors for non invasive analysis in precision agriculture,” *HardwareX*, vol. 20, p. e00596, 2024.
- [10] G. B. Romer and J. Delamoir, “The first color photographs,” *Scientific American*, vol. 261, no. 6, pp. 88–99, 1989.
- [11] H. Mannheimer, “The LANDSAT program,” in *Proceedings of the Conference on Waste Heat Management and Utilization*, vol. 2, NASA, 1977.
- [12] J. Y. Hardeberg, “Filter selection for multispectral color image acquisition,” pp. 105–110, 03 2004.
- [13] M. J. Vrhel and H. J. Trussell, “Filter considerations in color correction,” *IEEE Transactions on image processing*, vol. 3, no. 2, pp. 147–161, 1994.

- [14] Rosco Laboratories, “Rosco mycolor.” <https://legacy.rosco.com/mycolor/mycolor.cfm>, 2026. Accessed: 2026-03-29.
- [15] T. Smith and J. Guild, “The C.I.E. colorimetric standards and their use,” *Transactions of the Optical Society*, vol. 33, pp. 73–134, Jan. 1931.
- [16] A. Stockman, “Cone fundamentals and cie standards,” *Current Opinion in Behavioral Sciences*, vol. 30, pp. 87–93, 2019. Visual perception.
- [17] R. Von Luther, “Aus dem gebiet der farbreizmetrik,” *Zeitschrift fur Technishe Physik*, vol. 12, pp. 540–558, 1927.
- [18] Arducam, “1mp ov9281/ov9282 global shutter camera,” n.d. Accessed: 2026-04-13.
- [19] U.S. Geological Survey, “Landsat 1.” <https://www.usgs.gov/landsat-missions/landsat-1>, 2022. Accessed: 2025-05-20.
- [20] U.S. Geological Survey, “Landsat 1-5 multispectral scanner (MSS) image assessment system (IAS) radiometric algorithm description document (ADD),” Technical Report LS-IAS-07, European Space Agency (ESA) Earth Online, 2012. Accessed: 2025-05-20.
- [21] F. König and W. Praefcke, “Practice of multispectral image acquisition,” in *Electronic Imaging: Processing, Printing, and Publishing in Color*, vol. 3409, pp. 34–41, SPIE, 1998.
- [22] T. N. Greville, “Some applications of the pseudoinverse of a matrix,” *SIAM review*, vol. 2, no. 1, pp. 15–22, 1960.
- [23] F. J. Beutler, “The operator theory of the pseudo-inverse i. bounded operators,” *Journal of Mathematical Analysis and Applications*, vol. 10, no. 3, pp. 451–470, 1965.
- [24] R. MacAusland, “The moore-penrose inverse and least squares,” *Math 420: Advanced Topics in Linear Algebra*, vol. 551, pp. 1–10, 2014.
- [25] S. G. Mallat and Z. Zhang, “Matching pursuits with time-frequency dictionaries,” *IEEE Transactions on signal processing*, vol. 41, no. 12, pp. 3397–3415, 1993.
- [26] M. V. Lakshmi and J. R. Winkler, “Numerical properties of solutions of lasso regression,” *Applied Numerical Mathematics*, vol. 208, pp. 297–309, 2025. Special Volume on Numerical Analysis and Scientific Computation with Applications.
- [27] P. Mehta, M. Bukov, C.-H. Wang, A. G. Day, C. Richardson, C. K. Fisher, and D. J. Schwab, “A high-bias, low-variance introduction to machine learning for physicists,” *Physics Reports*, vol. 810, pp. 1–124, 2019. A high-bias, low-variance introduction to Machine Learning for physicists.
- [28] P. Vora and H. Trussell, “Mathematical methods for the design of color scanning filters,” *IEEE Transactions on Image Processing*, vol. 6, no. 2, pp. 312–320, 1997.
- [29] M. J. Vrhel, H. J. Trussell, and J. Bosch, “Design and realization of optimal color filters for multi-illuminant color correction,” *Journal of Electronic Imaging*, vol. 4, no. 1, pp. 6–14, 1995.
- [30] D.-Y. Ng and J. Allebach, “A subspace matching color filter design methodology for a multispectral imaging system,” *IEEE Transactions on Image Processing*, vol. 15, no. 9, pp. 2631–2643, 2006.

- [31] G. D. Finlayson and Y. Zhu, “Designing color filters that make cameras more colorimetric,” *IEEE Transactions on Image Processing*, vol. 30, pp. 853–867, 2020.
- [32] H. Gao, R. K. Mantiuk, and G. D. Finlayson, “Color-accurate camera capture with multispectral illumination and multiple exposures,” in *Computer Graphics Forum*, vol. 43, p. e15252, Wiley Online Library, 2024.
- [33] H. E. Ives, “The transformation of color-mixture equations from one system to another,” *Journal of the Franklin Institute*, vol. 180, no. 6, pp. 673–701, 1915.
- [34] P. Hanji, F. Zhong, and R. K. Mantiuk, “Noise-aware merging of high dynamic range image stacks without camera calibration,” in *European Conference on Computer Vision*, pp. 376–391, Springer, 2020.
- [35] A. O. Akyüz and E. Reinhard, “Noise reduction in high dynamic range imaging,” *Journal of Visual Communication and Image Representation*, vol. 18, no. 5, pp. 366–376, 2007.
- [36] S. Mann and R. Picard, “Beingundigital’with digital cameras,” *MIT Media Lab Perceptual*, vol. 1, no. 2, p. 3, 1994.
- [37] N. D. Sidiropoulos, J. S. Baras, and C. A. Berenstein, “Optimal filtering of digital binary images corrupted by union/intersection noise,” *IEEE transactions on image processing*, vol. 3, no. 4, pp. 382–403, 1994.
- [38] P.-J. Lapray, X. Wang, J.-B. Thomas, and P. Gouton, “Multispectral filter arrays: Recent advances and practical implementation,” *Sensors*, vol. 14, no. 11, pp. 21626–21659, 2014.
- [39] A. Morales, R. Guerra, P. Horstrand, M. Diaz, A. Jimenez, J. Melian, S. Lopez, and J. F. Lopez, “A multispectral camera development: From the prototype assembly until its use in a uav system,” *Sensors*, vol. 20, no. 21, p. 6129, 2020.
- [40] K. Martinez, J. Cupitt, and D. R. Saunders, “High-resolution colorimetric imaging of paintings,” in *Cameras, Scanners, and Image Acquisition Systems*, vol. 1901, pp. 25–36, SPIE, 1993.
- [41] Y. Yokoyama, N. Tsumura, H. Haneishi, Y. Miyake, J. Hayashi, and M. Saito, “A new color management system based on human perception and its application to recording and reproduction of art paintings,” in *Color and Imaging Conference*, vol. 5, pp. 169–172, Society of Imaging Science and Technology, 1997.
- [42] eoPortal Directory, “Iss utilization: HISUI (hyperspectral imager suite),” 2017. Last updated: September 15, 2017; Accessed: 2026-05-21.
- [43] E. R. H. Jr. and C. S. T. Daughtry, “What good are unmanned aircraft systems for agricultural remote sensing and precision agriculture?,” *International Journal of Remote Sensing*, vol. 39, no. 15-16, pp. 5345–5376, 2018.
- [44] S. Cadd, B. Li, P. Beveridge, W. T. OHare, and M. Islam, “Age determination of blood-stained fingerprints using visible wavelength reflectance hyperspectral imaging,” *Journal of Imaging*, vol. 4, no. 12, 2018.
- [45] S. Cadd, B. Li, P. Beveridge, W. T. OHare, A. Campbell, and M. Islam, “The non-contact detection and identification of blood stained fingerprints using visible wavelength reflectance hyperspectral imaging: Part 1,” *Science & Justice*, vol. 56, no. 3, pp. 181–190, 2016.

- [46] M. R. Almeida, L. P. L. Logrado, J. J. Zacca, D. N. Correa, and R. J. Poppi, “Raman hyperspectral imaging in conjunction with independent component analysis as a forensic tool for explosive analysis: The case of an atm explosion,” *Talanta*, vol. 174, pp. 628–632, 2017.
- [47] F. Yasuma, T. Mitsunaga, D. Iso, and S. K. Nayar, “Generalized assorted pixel camera: postcapture control of resolution, dynamic range, and spectrum,” *IEEE transactions on image processing*, vol. 19, no. 9, pp. 2241–2253, 2010.
- [48] A. Stockman and L. T. Sharpe, “Colour & vision research laboratory (cvrl).” <http://www.cvrl.org/>, n.d. Accessed: 2025-05-11.
- [49] D. Pascale, “Rgb coordinates of the macbeth colorchecker,” 2006.
- [50] X-Rite, “Colorchecker classic.” Accessed: 2026-05-21.
- [51] C. S. McCamy, H. Marcus, and J. G. Davidson, “A color-rendition chart,” *Journal of Applied Photographic Engineering*, vol. 2, no. 3, pp. 95–99, 1976.
- [52] P. Lindström, “Delta e blues: The science of color perception,” *Seybold Report: Analyzing Publishing Technologies*, vol. 8, no. 3, 2008.
- [53] M. R. Luo, “The new preferred memory color (pmc) chart,” *Color Research & Application*, vol. 49, no. 6, pp. 564–576, 2024.
- [54] Thousand Lights Lighting (Changzhou) Limited, “Preferred memory colour chart (PMCC).” https://thouslite.com/product_detail/1827.html, 2023. Accessed: 2025-05-15.
- [55] Coligh, “Shortpass filter ir cut 650nm,” n.d. Accessed: 2026-04-13.
- [56] Utopia Camera Services, “IANIRO LILIPUT 300/650w - 650/300W Open Face Tungsten Light.” <https://utopiacam.com/en/shop/liliput-300650w/>, n.d. Accessed: 2026-05-15.
- [57] W.-H. Yang, Y. Shao, Y.-W. Xu, *et al.*, “Guidelines on clinical research evaluation of artificial intelligence in ophthalmology (2023),” *International Journal of Ophthalmology*, vol. 16, no. 9, p. 1361, 2023.
- [58] J. Thompson, “Arducam case and mount for USB camera module.” <https://www.printables.com/model/141039-arducam-case-and-mount-for-usb-camera-module>, 2022. Accessed: 2026-05-29.
- [59] Stamos, “Raspberry Pi 5 case.” <https://www.printables.com/model/742926-raspberry-pi-5-case>, 2024. Accessed: 2026-05-29.
- [60] Baxcor, “MG996R servo model.” <https://www.printables.com/model/326713-mg996r-servo-model>, 2022. Accessed: 2026-05-29.
- [61] Prusa Research, “Original Prusa MK4S,” 2024. Accessed: 2026-05-29.
- [62] B. Kopová, M. Bakes, M. Čížek, A. Horký, J. Dvořák, K. Ráž, and Z. Chval, “Development and production of a children’s upper-limb cycling adapter using 3d printing,” *Materials*, vol. 17, no. 19, p. 4731, 2024.
- [63] A. M. Zeidan, Z. Xu, L. Leung, C. Byrne, S. Sabu, Y. Zhou, C. A. Rinaldi, J. Whitaker, S. E. Williams, J. Behar, A. Arujuna, R. J. Housden, and K. Rhode, “An anthropomorphic phantom for atrial transseptal puncture simulation training,” *3D Printing in Medicine*, vol. 10, 2024.

- [64] Raspberry Pi Ltd, “Raspberry Pi 5.” <https://thepihut.com/products/raspberry-pi-5>, 2023. Accessed: 2026-06-22.
- [65] Waveshare / Arducam, “OV9281 120-degree monochrome camera module.” <https://thepihut.com/products/ov9281-120-mono-camera>. Accessed: 2026-06-22.
- [66] TowerPro, “MG996R high torque metal gear servo motor.” <https://www.switchelectronics.co.uk/products/mg996r-metal-gear>. Accessed: 2026-06-22.
- [67] Rosco Laboratories Inc., “Rosco Supergel swatch book.” <https://stagedepot.co.uk/lighting/gel-kits-accessories/swatch-books/rosco-supergel-swatch-book>. Accessed: 2026-06-22.
- [68] J. Jiang, D. Liu, J. Gu, and S. Süssstrunk, “What is the space of spectral sensitivity functions for digital color cameras?,” in *IEEE Workshop on Applications of Computer Vision (WACV)*, pp. 168–179, 2013.
- [69] S. P. Farnand, “Using ΔE metrics for measuring color difference in hard copy pictorial images,” in *Color Imaging VIII: Processing, Hardcopy, and Applications*, vol. 5008, pp. 109–118, SPIE, 2003.
- [70] G. West and M. H. Brill, “Necessary and sufficient conditions for von kries chromatic adaptation to give color constancy,” *Journal of Mathematical Biology*, vol. 15, no. 2, pp. 249–258, 1982.
- [71] A. Stockman and L. T. Sharpe, “The spectral sensitivities of the middle- and long-wavelength-sensitive cones derived from measurements in observers of known genotype,” *Vision Research*, vol. 40, no. 13, pp. 1711–1737, 2000.

6.1 Appendix

6.1.1 Repository

The repository containing all code included in this research is available at this link:

<https://github.com/kylefram/MultispectralCamera/tree/main>

6.1.2 Capture Errors

A few captures in the lab featured artifacts, and a number of filters preserved transmission fairly uniformly across ω but included 'frost' or 'blur' effects which occasionally performed well in simulations but often added poor elements to reconstructions. A number of otherwise good captures included slight lens flare near the top of the capture or just above the car, these were included as they do not affect the measurement of the PMC. These filters numbered $\{0, 1, 100, 101, 104, 121, 122, 124, 125, 126, 127, 160, 61, 63\}$ were excluded from the main simulation set, and another round of simulations were run with the trimmed set. This had negligible effect on the quality of reconstruction.

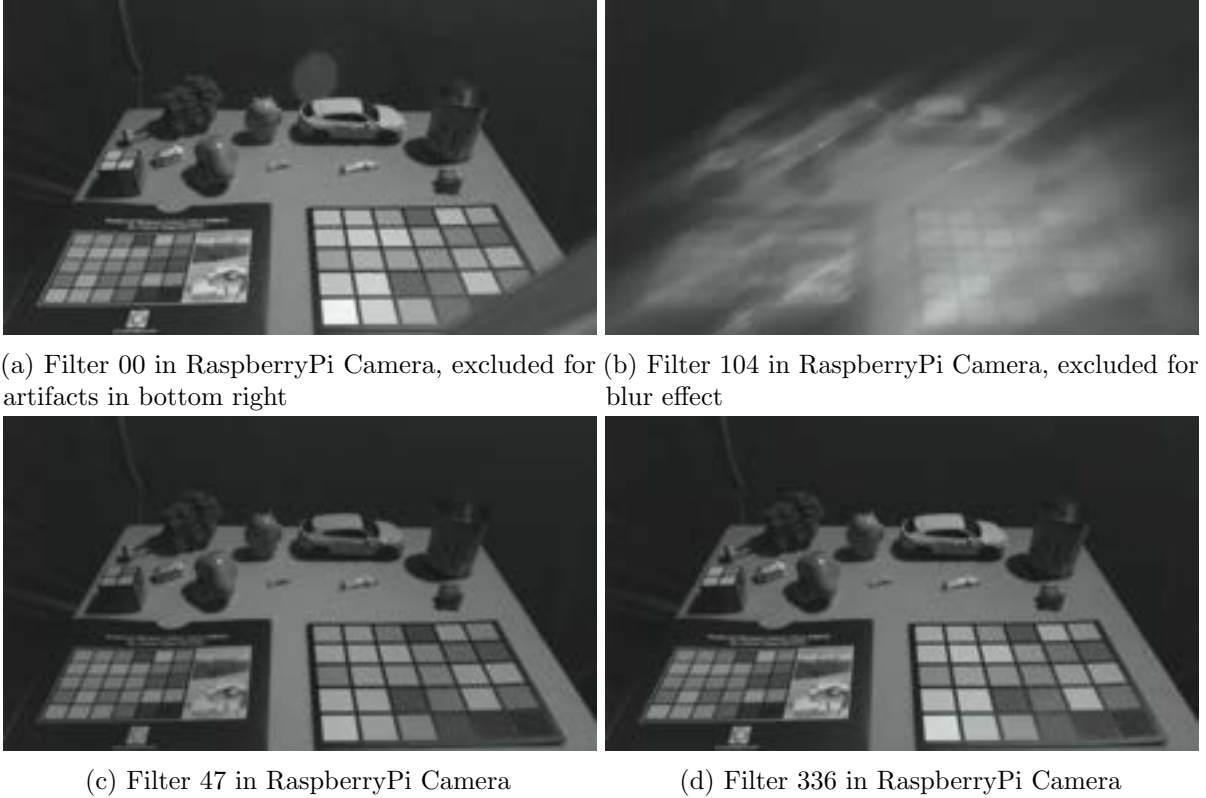


Figure 6.2: Monochrome Captures in Lab Environment

For demonstration, we show a reconstruction for the $k = 5$ case, where we simulate the optimal reconstruction including an IR-cutoff camera. However, the lab measurements were taken in the absence of an IR-cutoff camera, resulting in an ill-fitting reconstruction.

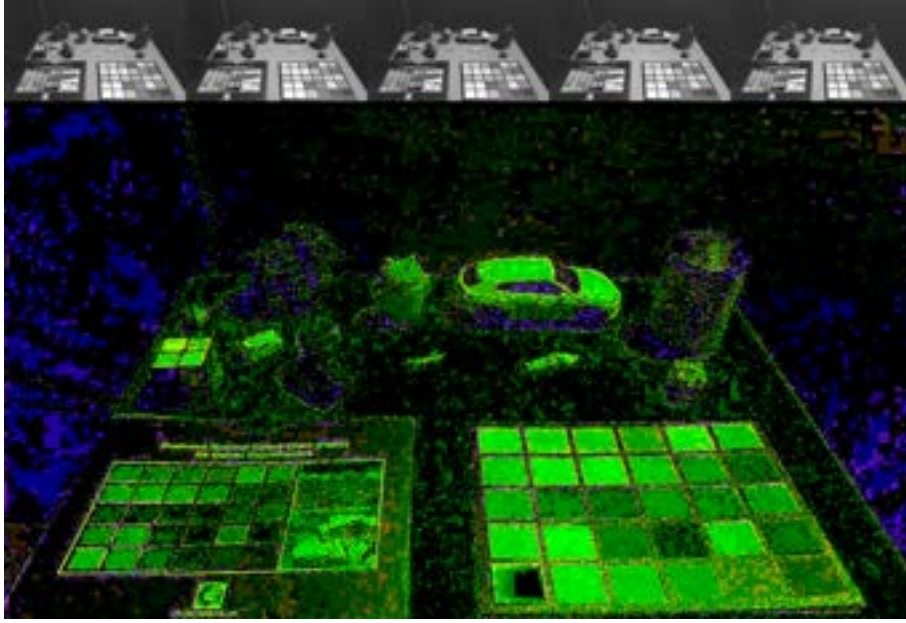
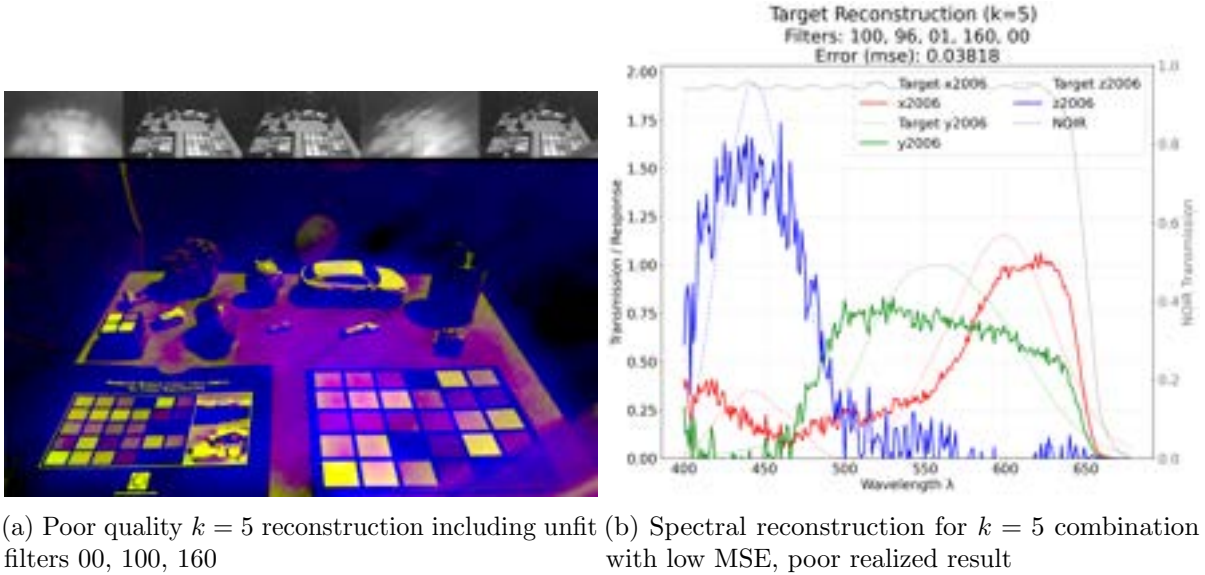


Figure 6.3: $k_1 = 5$ Photo Lab Reconstruction with component captures above, incorrect camera configuration

6.1.3 In-Lab Camera Tuning

We demonstrate a poorly tuned simulation, in which we run an optimal filter selection in the presence of an IR-cutoff filter, but our set of captures does not include this apparatus. This poor example shows a different $k = 5$ solution, but with the poor captures included. Capture 00 includes a blockage in the bottom right squares in the PMC, and is therefore unfit, while filters 100 and 160 have generally poor



(a) Poor quality $k = 5$ reconstruction including unfit filters 00, 100, 160 (b) Spectral reconstruction for $k = 5$ combination with low MSE, poor realized result

Figure 6.4: $k = 5$ filter combination with poor reconstruction quality

We also demonstrate a standard camera tuning methodology, where we minimize explicitly the average ΔE_{00} over the PMC. For $k = 3$, we achieve ΔE_{00} of 12, which is significantly above the just-noticeable-difference threshold and warrants further investigation of potential measurement errors. Either way, we use the simulated results for our main reconstruction efforts.

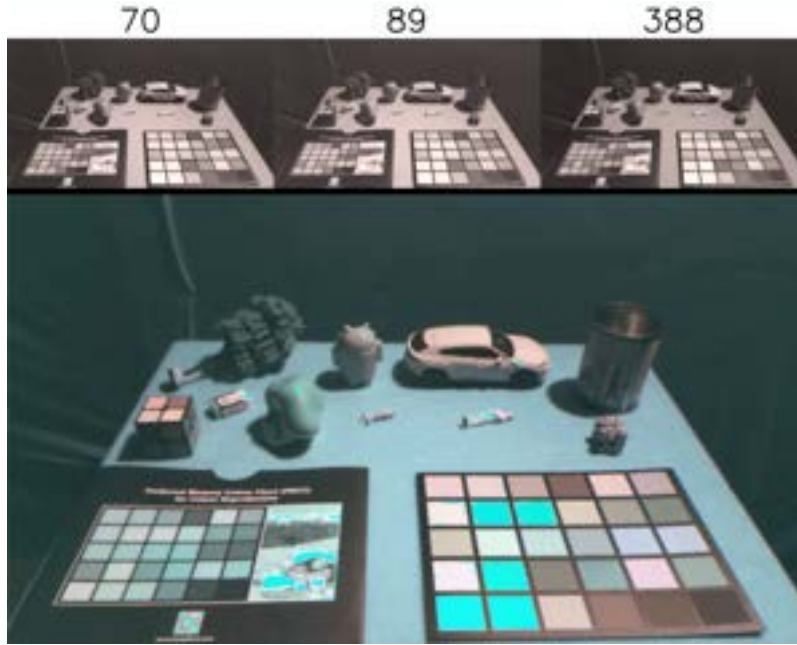


Figure 6.5: $k = 3$ reconstruction with ΔE_{00} minimization

6.1.4 Transmission Functions of Consumer Camera Market

In this section, we provide a few plots of spectral reconstruction functions vs. the CIE 2006 10° observer. We note that our reconstruction functions exhibit more continuity in spectral response, and generally match the human eye behavior more effectively. The data here is courtesy of Jiang et al. [68].

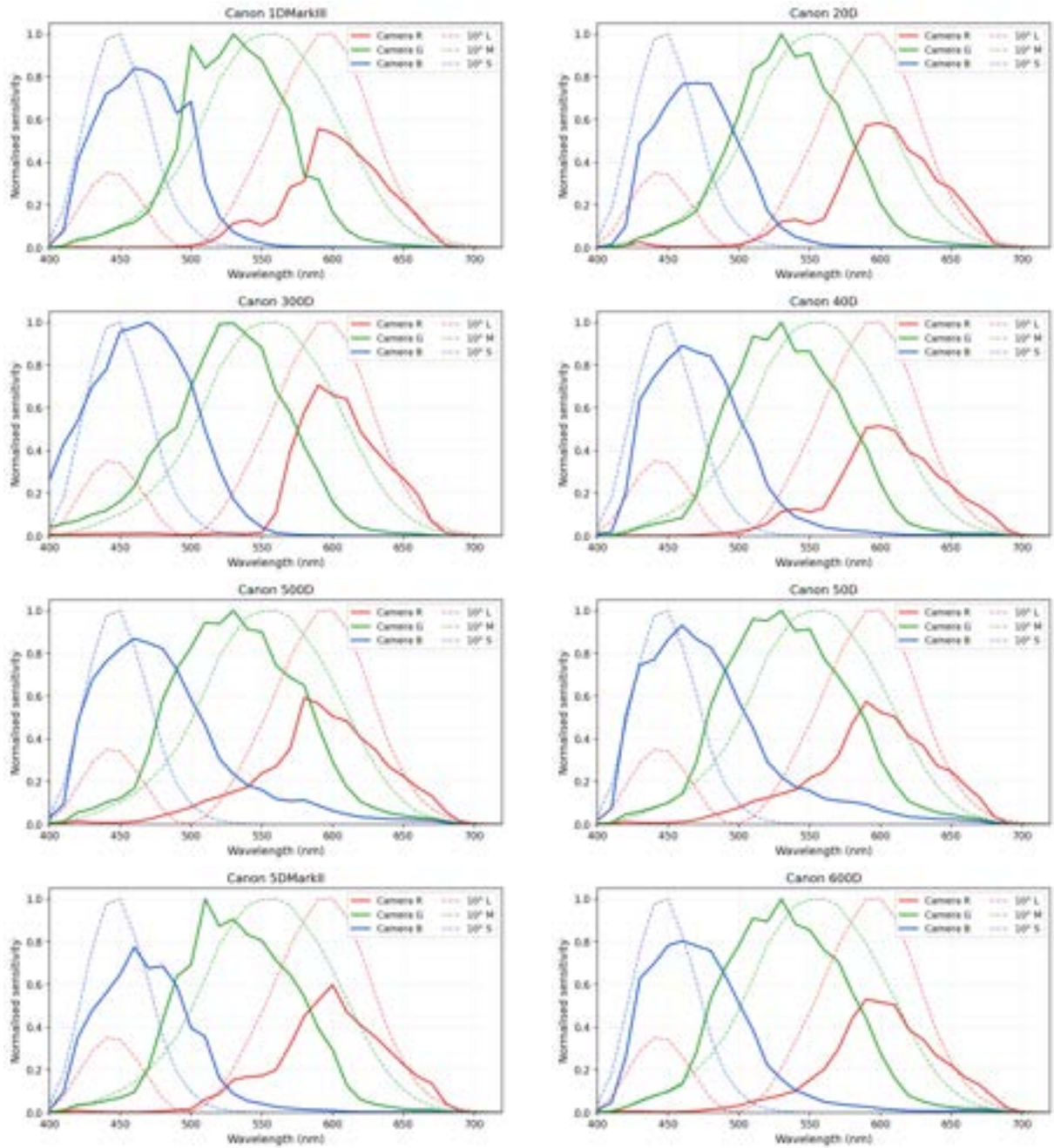


Figure 6.6: Spectral sensitivities (solid) of the 28 CamSpec cameras compared to the CIE 2006 10° LMS cone fundamentals (dashed), all normalised to a peak of 1. (1 of 3)

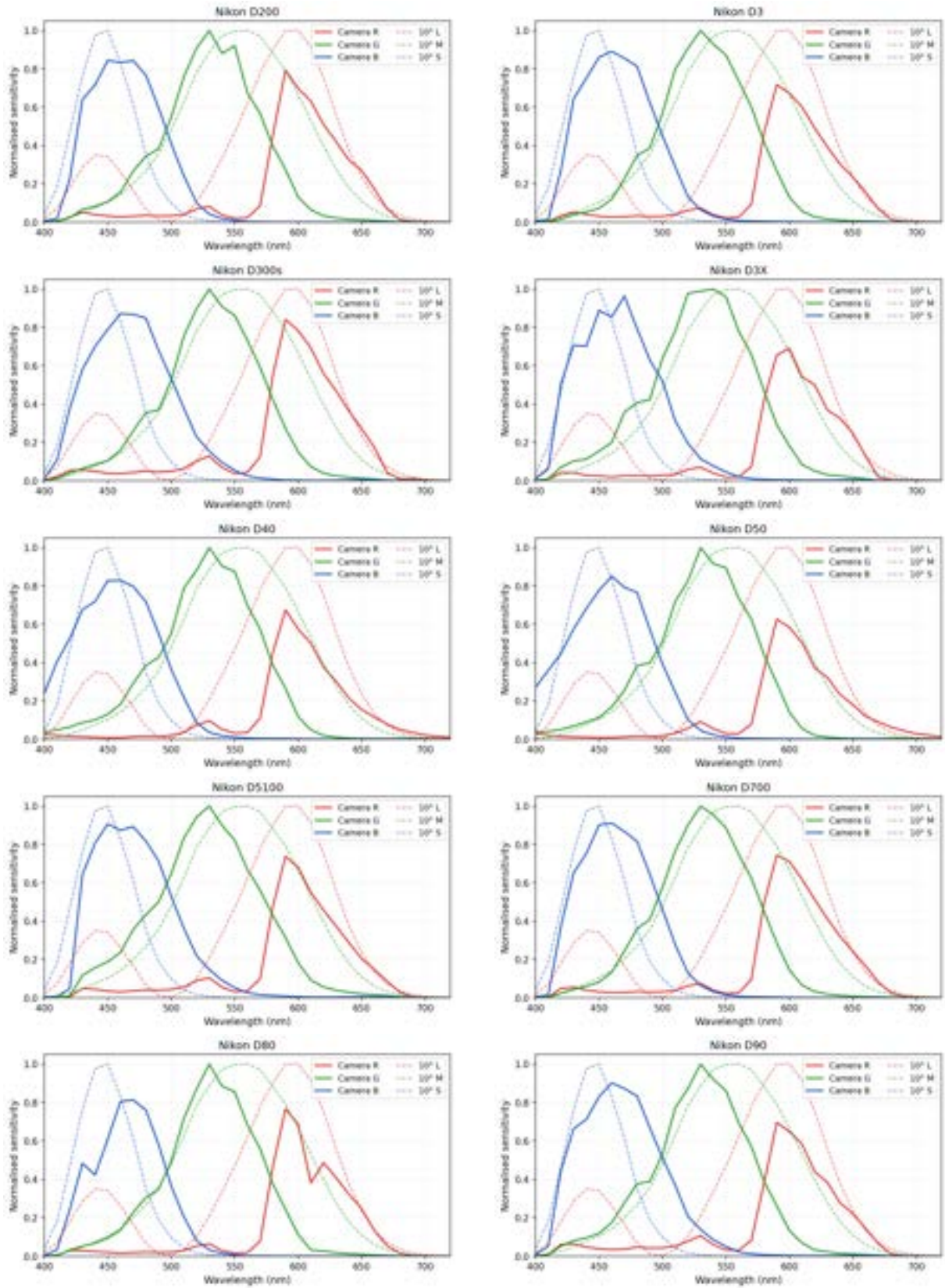


Figure 6.6: Spectral sensitivities (solid) of the 28 CamSpec cameras compared to the CIE 2006 10° LMS cone fundamentals (dashed), all normalised to a peak of 1. (2 of 3)

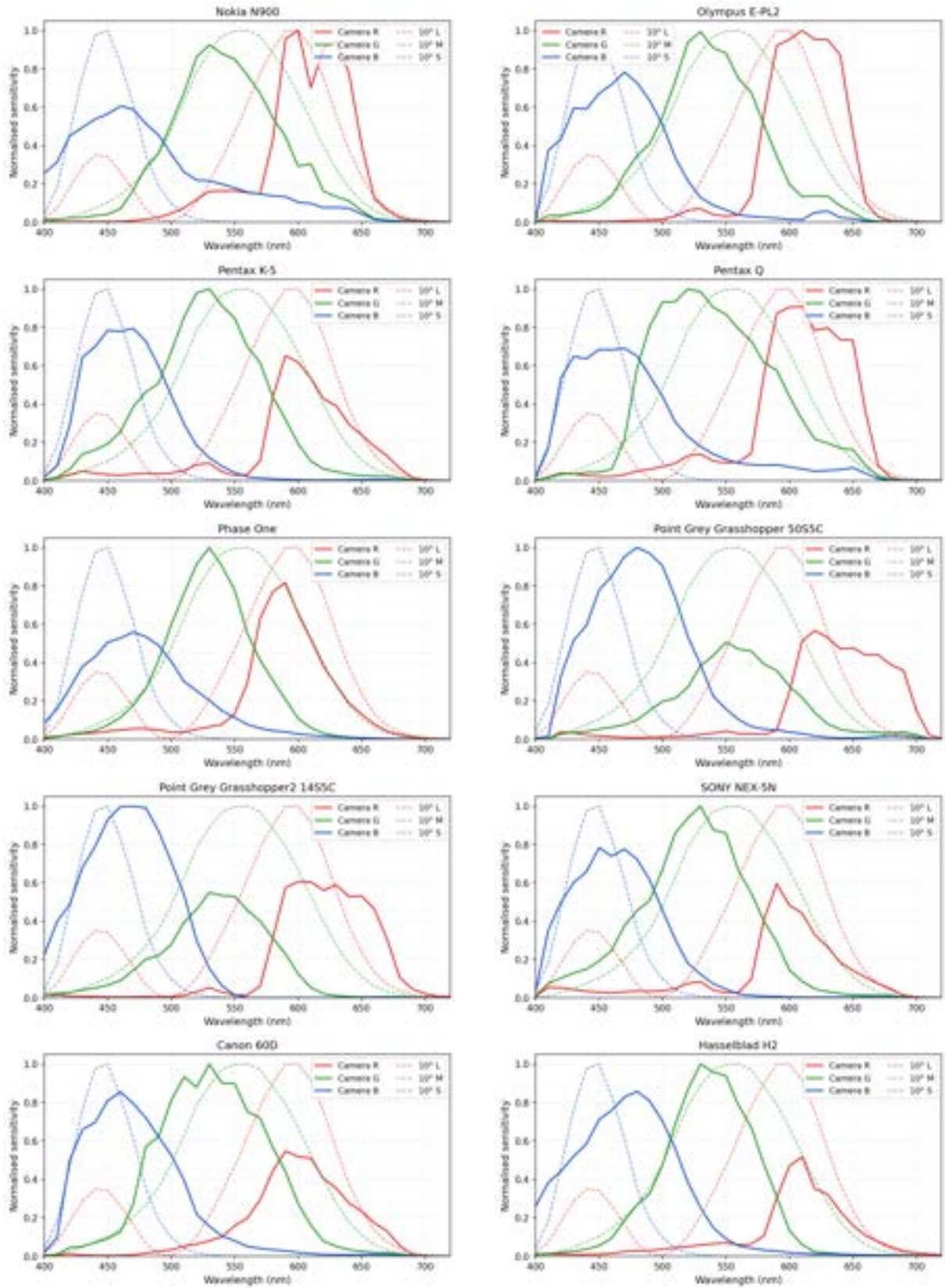


Figure 6.6: Spectral sensitivities (solid) of the 28 CamSpec cameras compared to the CIE 2006 10° LMS cone fundamentals (dashed), all normalised to a peak of 1. (3 of 3)